COMPLETE BUILD SPECIFICATION FOR AN AI CODING AGENT

PocketLedger

Expense and Budget Tracker

One document containing the App Brief, Product Requirements (PRD), Technical Requirements (TRD), App Flow, UI/UX Design Brief, Backend Schema and Implementation Plan. Give this PDF to an AI coding agent and it has everything needed to build the whole application without guessing.

Stack	PHP 8.3, Laravel (latest stable), MySQL 8, HTML (Blade templates), CSS (hand-written), TypeScript (Vite)
Platform	Responsive web application (desktop and mobile browsers)
Product	Personal income and expense tracking with monthly category budgets, recurring transactions and reports
Users	Individuals. One private account per person. No sharing in version 1
Version	1.0 (first usable version)
Date	17 October 2026

Contents

Instructions for the AI Coding Agent (Read First) 5

How to use this specification . . . 5
Environment assumptions . . . 5
Definition of done . . . 5
Naming rules (use these exact names everywhere) . . . 6

Document 0: App Brief 7

Document 1: Product Requirements Document (PRD) 8

1. Product Overview . . . 8
2. Target Users . . . 8
3. Core User Outcome . . . 8
4. Core Features . . . 8
5. Scope . . . 11
6. User Stories . . . 11
7. Functional Requirements . . . 13
8. Non-Functional Requirements . . . 14
9. Success Criteria . . . 14
10. Assumptions, Risks and Resolved Questions . . . 14

Document 2: Technical Requirements Document (TRD) 15

1. Technical Overview . . . 15
2. Technology Stack . . . 15
3. System Architecture . . . 16
4. APIs and Integrations . . . 18
5. Security and Privacy . . . 18
6. Performance Requirements . . . 18
7. Testing and Quality . . . 19
8. Development and Deployment . . . 19
9. Technical Risks and Open Questions . . . 19

Document 3: App Flow 20

1. Journey Summary (plain English) . . . 20
2. Entry Points . . . 20
3. Authentication Flow . . . 20
4. Screen Inventory . . . 21
5. Screen-by-Screen Behaviour . . . 23
6. Primary User Journey (happy path) . . . 27
7. Secondary Journeys . . . 27

8. Decision Points . . . 28
9. Edge Cases and Recovery . . . 28
10. Navigation Rules . . . 29
11. Flowchart (Mermaid) . . . 29

Document 4: UI/UX Design Brief **30**

1. Experience Goal . . . 30
2. Users and Context . . . 30
3. Visual Direction . . . 30
4. Layout System . . . 31
5. Component Language . . . 31
6. Screen Direction . . . 33
7. Interaction and Motion . . . 33
8. Accessibility . . . 33
9. Formatting Rules for Money and Dates . . . 34
10. Consistency Rules . . . 34

Document 5: Backend Schema **35**

1. Data Overview . . . 35
2. Authentication and Authorisation . . . 35
3. Tables . . . 35
4. Relationships and Delete Behaviour . . . 38
5. Access Rules . . . 38
6. Core Data Operations (SQL contracts) . . . 38
7. File Storage . . . 40
8. Data Integrity and Security . . . 40
9. Migrations, Factories and Seed Data . . . 40
10. Risks and Open Questions . . . 40

Entity-Relationship Diagram (Mermaid) . . . 41

Traceability: Table to PRD Feature . . . 41

Document 6: Implementation Plan **42**

1. Current Project State . . . 42
2. Build Principles . . . 42
3. Ordered Phases . . . 42

Phase 0: Project foundation and tooling . . . 42

Phase 1: Design system and layout . . . 43

Phase 2: Database schema, models and factories . . . 43

Phase 3: Authentication, settings and default categories . . . 44

Phase 4: Categories management . . . 44

Phase 5: Transactions . . . 45

Phase 6: Budgets . . . 45
Phase 7: Dashboard, reports, charts and CSV export . . . 46
Phase 8: Recurring transactions . . . 46
Phase 9: Polish, demo data, documentation and deployment readiness . . . 47
4. Suggested Sequence (summary) . . . 47
5. Requirement Traceability . . . 48
6. Final Verification . . . 48
7. Final Consistency Check (run before finishing) . . . 49

Appendix A: Setup, Configuration and CI **50**

Setup commands . . . 50
.env sample (local) . . . 50
vite.config.ts and tsconfig.json . . . 50
package.json scripts . . . 51
Scheduler registration (routes/console.php) . . . 51
GitHub Actions workflow (.github/workflows/ci.yml) . . . 52

Appendix B: Validation Rules **53**

Appendix C: JSON Endpoint Contracts **54**

Appendix D: Default Categories and Palette **55**

Appendix E: Test Inventory (minimum) **56**

Appendix F: Glossary **57**

Instructions for the AI Coding Agent (Read First)

This PDF is the single source of truth for the project. Build the **entire** application from an empty directory to a finished, tested, deployable state. Do not stop after the first phase to ask for approval; the specification below is already approved.

How to use this specification

1. Read the whole PDF before writing any code.
2. Each document is authoritative for its own area: **PRD** for product scope, **TRD** for technology and architecture, **App Flow** for routes, screens and states, **UI/UX Brief** for visual behaviour, **Backend Schema** for data, **Implementation Plan** for build order.
3. If two documents appear to conflict, follow the PRD for scope, the Backend Schema for table and column names, and the App Flow for routes. Record the conflict and your resolution in `docs/DECISIONS.md` and keep going.
4. Work through Implementation Plan phases 0 to 9 in order. After each phase run that phase's verification commands. Fix every failure before starting the next phase. Make one git commit per phase named `Phase N: <title>`.
5. Do not add features outside PRD section 5 (Included in version one). Do not change the approved stack. Do not add packages that are not listed in TRD section 2 unless a listed package cannot do the job; record any addition in `docs/DECISIONS.md`.
6. When something is not specified, choose the simplest option consistent with all documents, and record it in `docs/DECISIONS.md`. Only stop and ask the human if you are blocked by something you cannot create yourself (for example, no MySQL server reachable).
7. Copy this PDF into the project as `docs/SPEC.pdf`. Create `README.md` as described in Implementation Plan phase 9.

Environment assumptions

- PHP 8.3 or newer with extensions: `mbstring`, `intl`, `bcmath`, `pdo_mysql`, `xml`, `curl`, `zip`. Composer 2.
- Node.js 20 or newer (22 LTS preferred) with npm.
- MySQL 8.0 or newer reachable at `127.0.0.1:3306`. Create databases `pocketledger` and `pocketledger_test`. If Docker is available you may use Laravel Sail for MySQL.
- **Do not substitute SQLite or any other database**, including for tests. Several queries rely on MySQL behaviour (`DECIMAL`, `DATE_FORMAT`, `ENUM`, `ONLY_FULL_GROUP_BY`).
- Use `declare(strict_types=1);` in all PHP files you write. TypeScript runs in `strict` mode. Format PHP with Laravel Pint and keep PHPStan (Larastan) clean.

Definition of done

- Every PRD feature F1 to F8 works end to end in the browser and its acceptance criteria pass.
- All commands in Implementation Plan section 6 (Final Verification) pass with zero errors.
- The app can be started with documented commands, seeded with demo data and deployed using the README steps.
- No placeholder text, no TODO comments for required features, no unused routes, no dead code.

Naming rules (use these exact names everywhere)

Concept	Name in UI text	Model / table	Route prefix
A user account	Account	`User` / `users`	`/settings`
A kind of income or expense	Category	`Category` / `categories`	`/categories`
A recorded income or expense	Transaction	`Transaction` / `transactions`	`/transactions`
A monthly spending limit for one expense category	Budget	`Budget` / `budgets`	`/budgets`
A rule that creates transactions on a schedule	Recurring transaction	`RecurringTransaction` / `recurring_transactions`	`/recurring`

Document 0: App Brief

Source brief for every other document. Completed so that no product decision is left for the AI to guess.

App name	PocketLedger
One-line idea	A personal finance web app to record income and expenses, set monthly budgets per category, automate recurring transactions and see where the money goes.
Problem it solves	People lose track of spending during the month. Spreadsheets are tedious to maintain and do not warn when a category budget is about to run out.
Who it is for	Individuals (students and early-career professionals) managing their own money. Default currency is INR. One private account per person.
Main action	Add a transaction in under 15 seconds and immediately see the budget status of its category.
Must-have features	• Account registration, login, password reset, settings • Categories (income and expense) with archive • Transactions: add, edit, delete, filter, search, paginate • Monthly budgets per expense category with progress and status • Dashboard with monthly summary and charts • Reports: monthly summary, category breakdown, 6-month trend, top expenses, CSV export • Recurring transactions generated by a daily scheduled command
Nice-to-have (not in v1)	• Dark mode • CSV import • Tags and split transactions • Receipt uploads • Multi-currency conversion • Email budget alerts • REST API with Sanctum • Admin role
Platform	Web (responsive: desktop and mobile browsers)
Business model	None. Free portfolio and personal-use project.
Constraints	• Deadline: none fixed (suited to 2 to 3 weeks of part-time work) • Budget: zero. Open-source tools and free tiers only • Required tools: PHP, Laravel, MySQL, HTML (Blade), CSS (hand-written, no Tailwind or Bootstrap), TypeScript (Vite) • Privacy and security: hashed passwords, strict per-user data isolation, no third-party analytics or trackers
References	Envelope-style monthly budgeting as popularised by YNAB, used only as a concept. No visual copying.

Document 1: Product Requirements Document (PRD)

1. Product Overview

Product name	PocketLedger
Description	A web app for tracking personal income and expenses against monthly category budgets.
Problem being solved	Users cannot see, while they are spending, how much of each category budget is left.
Why it should exist	It makes logging fast, makes budget status visible at the moment of entry, and removes manual work for repeating items such as rent and salary.

2. Target Users

Primary user	A student or early-career professional in India who tracks personal money on a phone or laptop.
Current problem	Tracks spending in notes or spreadsheets, forgets entries, and finds out at month end that a category was overspent.
Desired outcome	Knows at any time what was spent this month per category versus plan, with minimal effort.
Secondary users	A reviewer (for example a recruiter) who opens the live demo account to evaluate the product. A seeded demo account serves this need.

3. Core User Outcome

At any moment the user can see how much they have spent this month versus what they planned, per category, and logging a new transaction takes only a few seconds.

4. Core Features

Eight features make up version one. IDs (F1 to F8) are used for traceability in every other document.

F1 Authentication and Account Settings

User need	Keep personal financial data private and let the user manage their account.
What it does	Register, sign in (optionally remember me), sign out, reset a forgotten password by email, update profile and currency, change password, delete account.
Inputs	Register: name, email, password, password confirmation, currency (default INR). Login: email, password, remember me. Settings: name, email, currency; current and new password; password to confirm deletion.
Expected output	Authenticated session. After registering, the user lands on the Dashboard with default categories already created.
Acceptance criteria	• Duplicate email is rejected with a field error • Wrong credentials show one generic message: "These credentials do not match our records." • More than 5 failed logins per minute for the same email and IP is throttled with a clear message • Password reset link works once and expires after 60 minutes • Changing password requires the current password • Deleting the account requires the password and permanently removes all of the user's data
Error and empty states	• Validation errors appear inline under each field and the form keeps the user's input (never the password) • Expired session shows the login page with the message "Your session expired. Please sign in again."

F2 Categories

User need	Group transactions in a way that matches how the user thinks about money.
What it does	Create, edit, archive, restore and (when unused) delete categories. Each category has a type (income or expense) and a colour.
Inputs	Name (max 60), type, colour (one of 8 preset colours).
Expected output	A list of categories grouped by type. Default categories are created at registration (see Appendix D).
Acceptance criteria	• Name is unique per user and type (case-insensitive) • Archived categories disappear from the pickers used to add or edit transactions, budgets and recurring rules, but still appear in history, reports and filters • A category cannot be deleted if it has transactions or recurring rules; the UI offers Archive instead • Type cannot be changed once the category has transactions, budgets or recurring rules
Error and empty states	• Empty state is not expected (defaults exist) but if all categories of a type are archived the pickers show "No active categories. Add one."

F3 Transactions

User need	Record money coming in and going out quickly and find past entries easily.
What it does	Add, edit and delete transactions. List them with filters, search and pagination, with totals for the filtered set.
Inputs	Category, amount, date, optional description. Filters: date range (default current month), type, category, text search.
Expected output	Saved transaction visible in the list. After saving an expense in a budgeted category a status message shows how much of that budget is used.
Acceptance criteria	• Amount: number from 0.01 to 999,999,999.99 with at most 2 decimals • Date: between 2000-01-01 and today (future dates are not allowed; recurring rules create future items when due) • Category must belong to the user and be active (an existing transaction may keep an archived category) • List shows 20 per page, newest first (date descending, then id descending) • Totals for income, expenses and net cover the whole filtered set, not only the current page • Delete asks for confirmation • Saved-expense message format: "Saved. {category}: {spent} of {limit} used in {Mon YYYY} ({pct}%)." when a budget exists
Error and empty states	• No results: "No transactions match these filters." with a Clear filters link • First use (no transactions at all): "No transactions yet. Add your first one." with an Add transaction button

F4 Budgets

User need	Plan spending per category and see progress against the plan.
What it does	Set a monthly limit for each expense category. Show spent, remaining, percent used and a status label. Copy limits from the previous month.
Inputs	Month (YYYY-MM) and a limit per expense category. Blank means no budget for that category.
Expected output	A single page listing every active expense category (plus archived ones that have a budget or spending in that month) with limit, spent, remaining, progress bar and status.
Acceptance criteria	• Limit: 0.01 to 999,999,999.99; blank removes the budget • Status by percent used: under 80% is "On track", 80% to 100% is "Nearing limit", over 100% is "Over budget" • Status is always shown as text and a bar, never colour alone • "Copy from previous month" fills only categories that have no budget in the selected month and never overwrites • Totals row: total budgeted, total spent in budgeted categories, total remaining • Selectable months: 2020-01 up to 12 months after the current month
Error and empty states	• No expense categories: link to Categories. No budgets yet: "No budgets set for this month. Enter a limit next to a category."

F5 Dashboard

User need	See the state of the month at a glance.
What it does	Show summary cards, spending by category chart, budget progress for top categories and recent transactions, with month navigation.
Inputs	Month (YYYY-MM), default current month.
Expected output	• Cards: Income, Expenses, Net, Budget used (expenses in budgeted categories divided by total of budget limits) • Donut chart of expenses by category (top 6 plus "Other") • Top 5 budgets by percent used • 8 most recent transactions
Acceptance criteria	• All figures use the selected month and the user's currency • Charts load from JSON endpoints and show a retry message if loading fails • Every chart has a text alternative (legend with values)
Error and empty states	• No transactions: "Nothing recorded for this month yet." with Add transaction button. No budgets: "No budgets set" with link to Budgets.

F6 Reports and CSV Export

User need	Understand spending patterns and take data out of the app.
What it does	Monthly summary, category breakdown with budget variance, 6-month income versus expense trend, top 5 largest expenses, and CSV export of transactions for a date range.
Inputs	Month for the reports; from and to dates for export.
Expected output	Reports page and a downloadable CSV file.
Acceptance criteria	• Savings rate = net divided by income as a percentage, shown as a dash when income is zero • Category breakdown lists spent, share of total expenses, budget limit and variance (limit minus spent) • Trend covers the selected month and the 5 months before it, with zero for empty months • CSV columns: date, type, category, amount, description. UTF-8 with BOM. Cells starting with = + - @ are prefixed with an apostrophe to prevent spreadsheet formula injection • Export range must be at most 5 years and the end date must not be before the start date
Error and empty states	• No data: "No data for this month." and the CSV button stays enabled (it exports an empty file with only the header)

F7 Recurring Transactions

User need	Stop re-entering rent, salary and subscriptions every month.
What it does	Create rules that automatically create transactions on a schedule (daily, weekly, monthly, yearly).
Inputs	Category, amount, description, frequency, start date (today or later), optional end date, active flag.
Expected output	Transactions appear in the normal list on their due dates, tagged as generated by a rule.
Acceptance criteria	• A scheduled command runs daily and creates all due transactions, including missed days (catch-up) • Running the command twice never creates duplicates • Monthly rules on day 29, 30 or 31 fall on the last day of shorter months and return to the original day when possible • A rule stops after its end date • Pausing a rule stops creation; resuming continues from the next due date on or after today • Editing a rule affects future occurrences only; deleting keeps already generated transactions • "Process due now" creates the current user's due transactions immediately and reports how many
Error and empty states	• No rules: "No recurring transactions. Add one for rent, salary or subscriptions."

F8 Data Privacy and Ownership

User need	Trust that nobody else can see or change the user's data.
What it does	Every record belongs to exactly one user. All queries and actions are restricted to the signed-in owner.
Inputs	None (system behaviour).
Expected output	Records of other users are never visible or reachable.
Acceptance criteria	• Requesting a record that belongs to another user returns 404 (not 403) • All state-changing requests require a CSRF token • Account deletion removes every row owned by the user
Error and empty states	• 419 page-expired is redirected to login with a message; 404 and 500 use branded error pages

5. Scope

Included in version one

- F1 to F8 as described above
- Single currency per user chosen from: INR, USD, EUR, GBP, AED, SGD, AUD, CAD (display only, no conversion)
- Responsive web UI, keyboard accessible, English only
- Demo seeder with 12 months of realistic data
- Automated tests, CI workflow and deployment instructions

Explicitly excluded from version one

- Multiple accounts or wallets, transfers between accounts
- Bank or UPI synchronisation, CSV import
- Receipt uploads or any file storage
- Shared, family or team accounts
- REST API, API tokens or a mobile app
- Admin panel or roles other than the account owner
- Email verification, push or email notifications
- Multi-currency conversion, tags, split transactions, savings goals
- Dark mode, internationalisation

Possible later additions

- Dark mode
- CSV import
- Email alerts when a budget reaches 80% and 100%
- Sanctum-based REST API
- Admin role with Gates and Policies
- Tags and goals
- Receipts
- Installable PWA

6. User Stories

1. As a new user, I want to register and immediately have sensible categories, so that I can log my first transaction right away.
2. As a user, I want to add an expense in a few seconds, so that I actually keep logging.

3. As a user, I want to see how much of a category budget I have used right after saving an expense, so that I can adjust my spending.
4. As a user, I want to filter and search my transactions, so that I can find a specific payment.
5. As a user, I want to set a monthly limit for each expense category, so that I can plan my spending.
6. As a user, I want to copy last month's budgets, so that I do not retype them every month.
7. As a user, I want a dashboard for the month, so that I can understand my situation at a glance.
8. As a user, I want a 6-month trend, so that I can see whether I am saving more or less over time.
9. As a user, I want my rent and salary to be created automatically, so that I do not forget them.
10. As a user, I want to export my transactions to CSV, so that I can analyse them elsewhere.
11. As a user, I want to archive a category I no longer use without losing history, so that reports stay accurate.
12. As a user, I want to delete my account and data, so that I stay in control of my privacy.

7. Functional Requirements

ID	Requirement	Feature
AUTH-01	Register with name, email, password (min 8, confirmed) and currency. In one database transaction create the user and the default categories, sign the user in and redirect to the Dashboard.	F1
AUTH-02	Sign in with email and password; optional remember me; regenerate the session on login; throttle to 5 failed attempts per minute per email and IP.	F1
AUTH-03	Sign out via POST, invalidate the session, redirect to login.	F1
AUTH-04	Forgot and reset password with a signed token valid for 60 minutes, single use. Mail is written to the log in local environments and sent by SMTP in production.	F1
AUTH-05	Settings: update name, email and currency; change password (requires current password); delete account (requires password) removing all owned data.	F1
CAT-01	Create and edit categories with name, type and colour. Unique per user and type.	F2
CAT-02	Archive and restore categories. Archived ones are excluded from entry pickers and kept in history.	F2
CAT-03	Delete a category only if it has no transactions and no recurring rules (its budgets are deleted with it).	F2
CAT-04	Prevent changing a category type when it has transactions, budgets or recurring rules.	F2
TXN-01	Create a transaction with category, amount, date and optional description, validated as in F3.	F3
TXN-02	Edit and delete a transaction. Deletion requires confirmation.	F3
TXN-03	List transactions with date range, type, category and text filters, paginated at 20, newest first.	F3
TXN-04	Show income, expense and net totals for the filtered set across all pages.	F3
TXN-05	After saving an expense, show the budget status message for its category and month when a budget exists.	F3, F4
BUD-01	Save budgets for a month in one bulk submit. Blank limit deletes the budget for that category.	F4
BUD-02	For each category show limit, spent, remaining, percent used and status text.	F4
BUD-03	Copy previous month's budgets into the selected month without overwriting existing ones.	F4
BUD-04	Show total budgeted, total spent in budgeted categories and total remaining.	F4
DASH-01	Dashboard cards: Income, Expenses, Net, Budget used for the selected month with previous and next month navigation.	F5
DASH-02	Dashboard donut chart of expenses by category (top 6 plus Other) with legend values.	F5
DASH-03	Dashboard list of the 5 budgets with the highest percent used and the 8 most recent transactions.	F5
RPT-01	Report: monthly summary with savings rate.	F6
RPT-02	Report: category breakdown with share, budget and variance.	F6
RPT-03	Report: 6-month income versus expense bar chart.	F6
RPT-04	Report: top 5 largest expenses of the month.	F6
RPT-05	CSV export for a date range, streamed, with formula-injection protection.	F6
REC-01	Create, edit and delete recurring rules.	F7
REC-02	Pause and resume a rule.	F7
REC-03	Daily scheduled command `recurring:process` creates due transactions idempotently with catch-up (maximum 400 occurrences per rule per run).	F7
REC-04	"Process due now" action for the signed-in user's rules.	F7
REC-05	Month-end clamping for monthly and yearly rules.	F7
REC-06	Rule list shows next run date and status (Active, Paused, Ended).	F7
OWN-01	All data access is scoped to the signed-in user; foreign records return 404.	F8

8. Non-Functional Requirements

Performance	Server-rendered pages respond in under 300 ms (p95, local machine) with 10,000 transactions for one user. All reports use SQL aggregation, never loops over all rows. JavaScript bundle under 150 KB gzipped.
Accessibility	Target WCAG 2.2 AA: contrast, keyboard operation, visible focus, labels, landmarks, reduced-motion support, 200% zoom.
Privacy	Store only name, email, hashed password, currency and the user's own financial entries. No analytics, trackers or third-party scripts. Do not log request bodies.
Security	CSRF protection, hashed passwords, rate limiting on login and reset, authorization policies, server-side validation of everything, escaped output, secure and HTTP-only cookies in production, HTTPS in production.
Browser and device support	Latest two versions of Chrome, Edge, Firefox and Safari. Mobile viewports from 360 px wide.

9. Success Criteria

- All acceptance criteria in section 4 pass, verified by automated tests where possible and the manual script in Implementation Plan section 6.
- Automated test suite, static analysis, type check and lint all pass in CI.
- Demo account with 12 months of data renders the Dashboard and Reports in under 1 second locally.
- Lighthouse accessibility score of 95 or higher on Dashboard, Transactions and Reports.

10. Assumptions, Risks and Resolved Questions

#	Decision or assumption	Why
A1	One currency per user, used for display only. Changing it does not convert stored amounts and the Settings page says so.	Keeps data model simple and honest.
A2	Transactions have a calendar date (no time of day). The application timezone is Asia/Kolkata.	Budgets and reports are monthly.
A3	Budgets exist only for expense categories.	Income does not need a spending limit.
A4	Future-dated transactions cannot be entered manually.	Avoids confusing month totals; recurring rules cover the planned future.
A5	No email verification in v1.	Reduces setup (mail) for a personal project.
A6	A transaction's type (income or expense) is determined by its category.	Avoids contradictory data.
A7	Archived categories keep working for existing recurring rules and history.	Prevents silent data loss.
R1	Risk: free hosts may not run the scheduler. Mitigation: "Process due now" button plus catch-up logic.	
R2	Risk: monetary rounding errors. Mitigation: DECIMAL(12,2) in the database, strings and bcmath in PHP, never floats.	

There are no unresolved product questions. Every decision needed to build version one is recorded in this document set.

Document 2: Technical Requirements Document (TRD)

Approved technical decisions. The coding agent must not change these without recording the reason in `docs/DECISIONS.md`.

1. Technical Overview

Architecture summary	Server-rendered monolith. Laravel handles routing, validation, business logic and HTML rendering with Blade. TypeScript (compiled by Vite) adds progressive enhancement and charts. MySQL stores all data. One deployable application, no separate API service, no queue workers.
Platforms	Web only. Responsive HTML and CSS.
Main constraints	• Stack fixed by the owner: PHP, Laravel, MySQL, HTML, CSS, TypeScript • No CSS framework and no JavaScript UI framework • Must run on free hosting tiers • Money must be exact (no floating point)

2. Technology Stack

Use the latest stable version of each tool at build time and pin exact versions through `composer.lock` and `package-lock.json`.

Layer	Choice	Reason	Rejected alternatives
Language and framework	PHP 8.3+, Laravel (latest stable)	Owner requirement; mature ecosystem; Eloquent, Form Requests, Policies, scheduler built in.	Symfony, CodeIgniter, plain PHP
Templating (HTML)	Blade views and Blade components	Server rendering keeps pages fast and simple; matches HTML requirement.	Inertia, Livewire, React or Vue SPA
Styling (CSS)	Hand-written modern CSS with custom properties, bundled by Vite	Owner wants CSS skills shown; small bundle; full control of the design system.	Tailwind, Bootstrap, Sass
Scripting	TypeScript 5 (strict) compiled by Vite with `laravel-vite-plugin`; no UI framework	Owner requirement; type safety for fetch and DOM code.	Plain JavaScript, Alpine.js, React
Charts	Chart.js 4 (tree-shaken imports, installed from npm)	Small, accessible enough with a text alternative, good TypeScript types.	D3 (too heavy), CDN script (external dependency)
Database	MySQL 8.0+ (InnoDB, utf8mb4, collation utf8mb4_unicode_ci)	Owner requirement; supports CHECK constraints and window-free aggregations needed here.	PostgreSQL, SQLite (not allowed, including tests)
Authentication	Laravel session guard with hand-written controllers and Form Requests	Full control over markup (no Tailwind kits) and flows required by F1.	Breeze, Jetstream, Fortify UI, Sanctum
Session, cache, queue	Session driver `database`, cache store `database`, queue connection `sync`	Works on any host without Redis or workers.	Redis, Horizon, queue workers
Scheduling	Laravel scheduler (`routes/console.php`) run by cron `* * * * * php artisan schedule:run`	Required for recurring transactions.	External cron services as the only mechanism

Layer	Choice	Reason	Rejected alternatives
Fonts	`@fontsource-variable/inter` and `@fontsource-variable/fraunces` (self-hosted)	No external requests, privacy friendly.	Google Fonts CDN
File storage	None	No uploads in version one.	S3, local disk uploads
Analytics	None	Privacy requirement.	Google Analytics, Plausible
PHP testing	Pest (with `pest-plugin-laravel`)	Readable tests, first-class Laravel support.	Plain PHPUnit
PHP quality	Laravel Pint (formatting), Larastan at level 6	Consistent style and static checks.	Psalm
TS testing and lint	Vitest with jsdom; ESLint with `typescript-eslint`; `tsc --noEmit`	Fast, Vite native.	Jest
Hosting	Local: Laravel Sail, Herd, Laragon or XAMPP. Production: Railway or Render with managed MySQL, or any VPS (Nginx, PHP-FPM, MySQL)	Free or low cost, supports cron or equivalent.	Serverless platforms without MySQL

3. System Architecture

Request flow: **Route** to **Controller** (thin) to **Form Request** (validation and authorisation of input) to **Action or Service** (business logic) to **Eloquent Model** to **MySQL**. A **Policy** guards every model access. The Controller returns a **Blade** view or a redirect. Two JSON endpoints feed charts.

Part	Responsibilities
Client (browser)	• Render HTML and CSS from the server • TypeScript: navigation drawer, flash dismiss, confirm dialogs, submit guard, transaction form category filtering, live budget totals, charts (fetch JSON and draw with Chart.js) • Never calculates authoritative money values; the server is the source of truth
Server (Laravel)	• Routing, sessions, CSRF, authentication, rate limiting • Validation and authorisation (Form Requests, Policies) • Business rules: budget status, recurrence, report aggregation, CSV streaming • Scheduled command for recurring transactions
Database (MySQL)	• Persistence, referential integrity, uniqueness and CHECK constraints • Aggregations for dashboard and reports (SUM, GROUP BY) • Indexes listed in the Backend Schema
External services	• SMTP mail provider, used only for password reset emails in production • Nothing else

Project layout

```
pocketledger/
  app/
    Actions/            RegisterUser, SeedDefaultCategories, DeleteUserAccount
    Console/Commands/   ProcessRecurringTransactions  (signature: recurring:process)
    Enums/              CategoryType, Frequency, BudgetStatus
    Http/Controllers/   Auth/*, DashboardController, TransactionController, CategoryController,
                        BudgetController, RecurringTransactionController, ReportController,
                        ReportDataController, SettingsController
    Http/Requests/      one Form Request per create/update action (see Appendix B)
    Models/             User, Category, Transaction, Budget, RecurringTransaction
    Policies/           CategoryPolicy, TransactionPolicy, RecurringTransactionPolicy
    Services/           BudgetStatusService, ReportService, RecurringTransactionProcessor,
                        RecurrenceCalculator, CurrencyFormatter, MonthRange
  bootstrap/app.php     routing, middleware, exception rendering
  database/             migrations, factories, seeders (DemoSeeder)
  resources/
    css/                app.css (imports tokens, base, layout, components, pages)
    ts/                 app.ts, types.ts, lib/, modules/, charts/
    views/              layouts, components, auth, dashboard, transactions, categories,
                        budgets, recurring, reports, settings, errors
  routes/               web.php, console.php
  tests/                Feature/, Unit/, ts/
  docs/                 SPEC.pdf, DECISIONS.md
  .github/workflows/    ci.yml
```

TypeScript modules

File	Purpose
`ts/app.ts`	Entry point. Imports fonts and CSS, then calls each module's `init()` when its `data-` hook exists on the page.
`ts/types.ts`	Shared types: `CategoryBreakdownResponse`, `MonthlyTrendResponse`, `CategoryType`.
`ts/lib/money.ts`	`formatMoney(amount, currency)` using `Intl.NumberFormat` (locale `en-IN` for INR, `en-US` otherwise).
`ts/lib/dom.ts`	Typed helpers `qs`, `qsa`, `on`.
`ts/modules/nav-drawer.ts`	Opens and closes the mobile navigation drawer; Escape closes; manages `aria-expanded`.
`ts/modules/flash.ts`	Dismiss buttons for flash messages.
`ts/modules/confirm-dialog.ts`	Forms with `data-confirm="..."` open a native `<dialog>` for confirmation before submit.
`ts/modules/submit-guard.ts`	Disables the submit button and sets `aria-busy` after submit to prevent double submits.
`ts/modules/transaction-form.ts`	Type toggle (Expense or Income) filters category options by `data-type`.
`ts/modules/budget-form.ts`	Recomputes the live total of budget limits while typing.
`ts/charts/chart-loader.ts`	Fetches chart JSON, shows loading and retry states, respects reduced motion.
`ts/charts/category-donut.ts`, `ts/charts/monthly-trend.ts`	Draw the donut and the bar chart with Chart.js (import only the controllers and elements used).

4. APIs and Integrations

Integration	Details
Internal JSON: category breakdown	`GET /reports/data/category-breakdown?month=YYYY-MM`. Auth: session cookie. Returns amounts as strings. Failure: HTTP 401/404/500 with JSON `{"message": "..."}`; the chart shows a retry button. Contract in Appendix C.
Internal JSON: monthly trend	`GET /reports/data/monthly-trend?month=YYYY-MM`. Same rules. Contract in Appendix C.
SMTP mail	Purpose: password reset emails. Data sent: recipient address and reset link. Auth: `MAIL_*` environment variables. No usage limit assumptions beyond the provider. Failure: log the error, show the generic message "If that email exists, a reset link has been sent." Local default: `MAIL_MAILER=log`.

5. Security and Privacy

- **Authentication**: Laravel `web` guard, session driver `database`, session lifetime 120 minutes, regenerate session on login, invalidate and regenerate token on logout. Remember me uses the default Laravel cookie.
- **Authorisation**: Policies for Category, Transaction and RecurringTransaction compare `user_id` to the signed-in user. Use `Gate::denyAsNotFound()` or route-model-binding scoped to the user so foreign records return 404.
- **Budgets** are always read and written through `$request->user()->budgets()` so they are scoped by construction.
- **Validation**: every input goes through a Form Request. Never trust hidden fields (for example category ownership is always re-checked on the server).
- **Output escaping**: use `{{ }}` only. Do not use `{!! !!}` except for trusted inline SVG icons from the project's own icon component.
- **CSRF**: `@csrf` on every form; fetch calls are GET only.
- **Rate limiting**: login 5 attempts per minute per email plus IP; password reset request 3 per minute per IP (named limiters).
- **Mass assignment**: use `$fillable` lists on every model. `user_id` is never fillable; it is set from the authenticated user.
- **Secrets**: only in `.env`; `.env` is git-ignored; provide `.env.example` without secrets. Production sets `APP_DEBUG=false`, `SESSION_SECURE_COOKIE=true`.
- **Headers**: add a middleware that sends `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin` and a Content-Security-Policy allowing only same-origin scripts, styles and fonts (plus `data:` images).
- **Sensitive data**: store only what F1 lists. Passwords are hashed with Laravel's default hasher via the `hashed` cast. Do not log request bodies or amounts.
- **CSV**: neutralise formula injection (prefix cells starting with `=`, `+`, `-`, `@`, tab or carriage return with an apostrophe).

6. Performance Requirements

Expected usage	Tens to hundreds of users; up to 10,000 transactions per user.
Loading targets	p95 server time under 300 ms for Dashboard and Reports with 10,000 rows (local). JS bundle under 150 KB gzipped. No layout shift from fonts (`font-display: swap` and preload of the main font file).
Caching approach	No application data caching. In production run `config:cache`, `route:cache`, `view:cache`, `event:cache`. Use Vite hashed asset filenames.
Database	Aggregate in SQL. Eager load relations (`with('category')`) to avoid N+1 queries. Add the indexes in the Backend Schema. In tests assert query counts for list pages stay constant regardless of row count.
Media optimisation	No images. Icons are inline SVG. Fonts are latin-subset woff2 files from Fontsource.

7. Testing and Quality

Type	Scope and tools
Unit (Pest)	`RecurrenceCalculator` (all frequencies, month-end clamping, leap years), `MonthRange`, `BudgetStatusService` thresholds, `CurrencyFormatter`.
Feature (Pest, MySQL)	Every route: guest redirect, happy path, validation failures, ownership (404 on foreign records). Registration seeds categories. Budget copy does not overwrite. Recurring processing is idempotent. CSV content and injection protection. Report queries against known fixtures.
TypeScript (Vitest)	`formatMoney`, category filtering function of the transaction form, chart data mapping functions.
Static analysis	`tsc --noEmit`, ESLint, Larastan level 6, Pint in `--test` mode.
End-to-end	No automated browser tests in v1. Use the manual script in the Implementation Plan.
Accessibility	Manual keyboard pass and an axe browser extension scan on every screen. Lighthouse accessibility 95 or higher.

8. Development and Deployment

Environments	• local (`APP_ENV=local`, `MAIL_MAILER=log`) • testing (`pocketledger_test` database, `MAIL_MAILER=array`) • production (`APP_ENV=production`, `APP_DEBUG=false`) • A separate staging environment is not required
Environment variables	See the `.env` sample in Appendix A. Production must also set `APP_KEY`, `APP_URL`, `DB_*`, `MAIL_*`, `SESSION_SECURE_COOKIE=true`.
CI checks	GitHub Actions workflow in Appendix A: type check, lint, TS tests, front-end build, Pint, Larastan and Pest against a MySQL 8 service.
Deployment approach	• `composer install --no-dev --optimize-autoloader` • `npm ci && npm run build` • `php artisan migrate --force` • `php artisan config:cache && php artisan route:cache && php artisan view:cache && php artisan event:cache` • Cron: `* * * * * php artisan schedule:run >> /dev/null 2>&1` • Serve the `public/` directory only

9. Technical Risks and Open Questions

Risk	Handling
Money arithmetic	Amounts are `DECIMAL(12,2)` and travel through PHP as strings. Use `bcadd`, `bcsub`, `bcmul`, `bcdiv` (scale 2 or higher for percentages). Never cast to float. Eloquent cast: `decimal:2`.
`ONLY_FULL_GROUP_BY`	MySQL 8 enables it by default. Every selected column in aggregate queries must be in `GROUP BY` or be aggregated. Queries in the Backend Schema already comply.
Month-end recurrence	Compute occurrence n from `start_date` (not from the previous occurrence) using Carbon `addMonthsNoOverflow` and `addYearsNoOverflow`.
Scheduler on free hosts	Catch-up logic plus the "Process due now" button.
Foreign key order on account deletion	`categories` are referenced with `RESTRICT` by transactions and recurring rules, so `DeleteUserAccount` deletes rows explicitly in a safe order inside one DB transaction (see Backend Schema section 8).
Chart accessibility	Provide legends with values and, on Reports, a data table next to each chart.
Open questions	None. All requirements can be satisfied by this stack.

Document 3: App Flow

Every screen, route, decision and state. Route names are the Laravel route names to register; URLs are exact.

1. Journey Summary (plain English)

A new visitor lands on the login page and chooses Register. After registering they arrive on the Dashboard, which is empty and invites them to add a transaction. They add an expense; the app confirms it and shows how much of that category's budget is used (if a budget exists). They open Budgets to set monthly limits, Recurring to automate rent and salary, and later use Dashboard and Reports to review the month. Returning users go straight to the Dashboard.

2. Entry Points

First visit	`GET /` redirects guests to `/login` (with a "Create an account" link). There is no marketing landing page.
Returning user	`GET /` redirects authenticated users to `/dashboard`. Opening `/login` or `/register` while signed in redirects to `/dashboard`.
Shared or deep link	Data is private, so there are no public share links. A deep link to any protected URL while signed out redirects to `/login` and, after login, returns to the originally requested URL (intended URL).

3. Authentication Flow

Step	Behaviour
Sign up	`/register` form. On success (AUTH-01): create user and default categories, sign in, redirect to `/dashboard` with the flash "Welcome to PocketLedger. Your categories are ready."
Sign in	`/login` form. On success: redirect to intended URL or `/dashboard`. On failure: stay on page, show "These credentials do not match our records." and keep the email. After 5 failures in a minute: "Too many attempts. Try again in N seconds."
Verification	Not used in version one.
Password recovery	`/forgot-password` accepts an email and always shows "If that email exists, a reset link has been sent." The email link opens `/reset-password/{token}?email=...`. On success redirect to `/login` with "Your password has been reset." Invalid or expired token shows an error on the reset form with a link to request a new one.
Sign out	Button in the sidebar user area submits `POST /logout`; redirect to `/login`.

4. Screen Inventory

#	Screen	Route name	URL	Access
1	Login	`login`	`GET /login`	Guest
2	Register	`register`	`GET /register`	Guest
3	Forgot password	`password.request`	`GET /forgot-password`	Guest
4	Reset password	`password.reset`	`GET /reset-password/{token}`	Guest
5	Dashboard	`dashboard`	`GET /dashboard`	Auth
6	Transactions list	`transactions.index`	`GET /transactions`	Auth
7	Add transaction	`transactions.create`	`GET /transactions/create`	Auth
8	Edit transaction	`transactions.edit`	`GET /transactions/{transaction}/edit`	Auth, owner
9	Categories list	`categories.index`	`GET /categories`	Auth
10	Add category	`categories.create`	`GET /categories/create`	Auth
11	Edit category	`categories.edit`	`GET /categories/{category}/edit`	Auth, owner
12	Budgets	`budgets.index`	`GET /budgets?month=YYYY-MM`	Auth
13	Recurring list	`recurring.index`	`GET /recurring`	Auth
14	Add recurring	`recurring.create`	`GET /recurring/create`	Auth
15	Edit recurring	`recurring.edit`	`GET /recurring/{recurringTransaction}/edit`	Auth, owner
16	Reports	`reports.index`	`GET /reports?month=YYYY-MM`	Auth
17	Settings	`settings.edit`	`GET /settings`	Auth
18	Error pages	(exceptions)	404, 419, 429, 500, 503	Any

Action routes (non-screen)

Method and URL	Route name	Result
`POST /login`	`login.store`	Sign in
`POST /register`	`register.store`	Create account
`POST /logout`	`logout`	Sign out
`POST /forgot-password`	`password.email`	Send reset link
`POST /reset-password`	`password.update`	Set new password
`POST /transactions`	`transactions.store`	Create transaction
`PUT /transactions/{transaction}`	`transactions.update`	Update transaction
`DELETE /transactions/{transaction}`	`transactions.destroy`	Delete transaction
`POST /categories`	`categories.store`	Create category
`PUT /categories/{category}`	`categories.update`	Update category
`DELETE /categories/{category}`	`categories.destroy`	Delete unused category
`PATCH /categories/{category}/archive`	`categories.archive`	Archive
`PATCH /categories/{category}/restore`	`categories.restore`	Restore
`PUT /budgets`	`budgets.update`	Bulk save budgets for a month
`POST /budgets/copy-previous`	`budgets.copy`	Copy previous month limits
`POST /recurring`	`recurring.store`	Create rule
`PUT /recurring/{recurringTransaction}`	`recurring.update`	Update rule
`DELETE /recurring/{recurringTransaction}`	`recurring.destroy`	Delete rule
`PATCH /recurring/{recurringTransaction}/toggle`	`recurring.toggle`	Pause or resume
`POST /recurring/process-due`	`recurring.process`	Create due transactions now
`GET /reports/export`	`reports.export`	CSV download (from, to)
`GET /reports/data/category-breakdown`	`reports.data.category-breakdown`	JSON for donut and table
`GET /reports/data/monthly-trend`	`reports.data.monthly-trend`	JSON for trend chart
`PUT /settings/profile`	`settings.profile`	Update name, email, currency
`PUT /settings/password`	`settings.password`	Change password
`DELETE /settings/account`	`settings.destroy`	Delete account and data

> Register resources with `Route::resource(...)->except(['show'])`. For recurring use `Route::resource('recurring', RecurringTransactionController::class)->parameters(['recurring' => 'recurringTransaction'])->except(['show'])`. Route-model binding must be scoped to the signed-in user so a foreign id returns 404.

5. Screen-by-Screen Behaviour

Login (login, GET /login)

Access	Guests only
Purpose	Let a returning user sign in.
Reached from	Redirect from `/`, from protected URLs, after logout or password reset.
Main content	Email, password, "Remember me", Sign in button, links to Forgot password and Create account.
Primary and secondary actions	Primary: Sign in. Secondary: Forgot password, Create account.
After each action	Success: intended URL or Dashboard. Failure: re-render with error and kept email.
States and edge cases	• Loading: button disabled with "Signing in..." after submit • Errors: invalid credentials, throttled, validation • Already signed in: redirect to Dashboard

Register (register, GET /register)

Access	Guests only
Purpose	Create an account.
Reached from	Login screen link.
Main content	Name, email, password, confirm password, currency select (default INR), Create account button.
Primary and secondary actions	Primary: Create account. Secondary: link to Sign in.
After each action	Success: Dashboard with welcome flash. Failure: inline field errors.
States and edge cases	• Duplicate email error • Password under 8 characters error • Password never repopulated

Forgot and Reset password (password.request / password.reset, /forgot-password, /reset-password/{token})

Access	Guests only
Purpose	Recover access.
Reached from	Login screen link; email link.
Main content	Email field (request) or email, new password, confirmation (reset).
Primary and secondary actions	Primary: Send reset link or Reset password. Secondary: Back to sign in.
After each action	Request: neutral success message. Reset success: redirect to login with message.
States and edge cases	• Expired or used token error with link to request again • Throttled: 429 page or inline message

Dashboard (dashboard, GET /dashboard?month=YYYY-MM)

Access	Authenticated
Purpose	Summarise the selected month.
Reached from	Sidebar, redirect after login, logo link, header Add transaction button.
Main content	• Month switcher (previous, current label, next, "This month") • Cards: Income, Expenses, Net, Budget used • Donut chart of expenses by category with legend values • Budgets list (top 5 by percent used, with status text) • Recent transactions (8) with link "View all"
Primary and secondary actions	Primary: Add transaction. Secondary: month switcher, View all transactions, Manage budgets, row click opens the edit form.
After each action	Month switch reloads the page with `?month=`. Add transaction goes to `transactions.create`.
States and edge cases	• Empty month: "Nothing recorded for this month yet." with Add transaction button • No budgets: "No budgets set" with link to Budgets • Chart fetch failure: card shows "Could not load chart" and a Retry button • Invalid `month` value falls back to the current month

Transactions list (transactions.index, GET /transactions)

Access	Authenticated
Purpose	Browse, filter and manage transactions.
Reached from	Sidebar, Dashboard "View all", redirect after save or delete.
Main content	• Filter bar: from, to (default current month), type, category, search text, Apply, Clear • Totals strip: Income, Expenses, Net for the filtered set • Ruled list: date, description (or category if empty), category with colour dot, amount; recurring-generated rows show a "Recurring" label • Pagination (20 per page, query string kept)
Primary and secondary actions	Primary: Add transaction. Secondary: Edit (row link), Delete (opens confirm dialog), filters.
After each action	Delete confirm submits `DELETE`, then redirects back to the list with "Transaction deleted." Edit goes to `transactions.edit` with a `return_to` value.
States and edge cases	• No results and first-use states from PRD F3 • Invalid filter values are ignored (default applied) • Page beyond last page redirects to the last page

Add and Edit transaction (transactions.create / transactions.edit, /transactions/create, /transactions/{transaction}/edit)

Access	Authenticated, owner for edit
Purpose	Create or change a transaction.
Reached from	Add transaction buttons, list row Edit link.
Main content	• Type toggle (Expense, Income; client-side only, filters the category list) • Category select (grouped by type without JavaScript, filtered by the toggle with JavaScript) • Amount (decimal keyboard), Date (default today), Description (optional) • On edit: hidden `return_to`
Primary and secondary actions	Primary: Save. Secondary: Cancel (returns to list), Save and add another (create only).
After each action	Create success: redirect to list (or back to the form for Save and add another) with the saved message including budget status when applicable. Update success: redirect to `return_to` if it starts with `/transactions`, else the list, with "Transaction updated."
States and edge cases	• Validation errors inline • No active categories of the chosen type: message with link to Categories • Editing a transaction whose category is archived: that category stays selectable

Categories (categories.index / create / edit, /categories, /categories/create, /categories/{category}/edit)

Access	Authenticated, owner for edit
Purpose	Manage categories.
Reached from	Sidebar; link from empty-picker messages.
Main content	• Two sections: Expense and Income, each a ruled list with colour dot, name, transaction count, status (Active or Archived) • Toggle "Show archived" • Form: name, type (read-only once the category is used), colour swatches (8 presets)
Primary and secondary actions	Primary: Add category. Secondary: Edit, Archive or Restore, Delete (only when unused).
After each action	Archive and restore are PATCH actions that redirect back with a message. Delete asks for confirmation.
States and edge cases	• Delete blocked when used: button hidden and server returns an error message "This category has transactions or recurring rules. Archive it instead." • Duplicate name error inline

Budgets (budgets.index, GET /budgets?month=YYYY-MM)

Access	Authenticated
Purpose	Set and review monthly limits.
Reached from	Sidebar; Dashboard link.
Main content	• Month switcher • Totals: budgeted, spent in budgeted categories, remaining • One row per expense category: name, limit input, spent, remaining, progress bar, status text • Copy from previous month button
Primary and secondary actions	Primary: Save budgets (single form, `PUT /budgets`). Secondary: Copy from previous month (`POST /budgets/copy-previous`), month switcher.
After each action	Save: redirect to the same month with "Budgets saved." Copy: redirect with "Copied N budgets." (or "Nothing to copy.").
States and edge cases	• Unsaved changes are lost when switching month (the switcher is a link; no autosave) • Live total of limits updates while typing (TypeScript) • Invalid amounts show inline errors on the row

Recurring (recurring.index / create / edit, /recurring, /recurring/create, /recurring/{recurringTransaction}/edit)

Access	Authenticated, owner for edit
Purpose	Automate repeating transactions.
Reached from	Sidebar.
Main content	• List: description, category, amount, frequency, next run, status (Active, Paused, Ended) • Form: category, amount, description, frequency, start date, end date, active checkbox
Primary and secondary actions	Primary: Add recurring transaction. Secondary: Edit, Pause or Resume, Delete, Process due now.
After each action	Toggle and process-due redirect back with messages ("Rule paused.", "Created 3 transactions." or "Nothing is due.").
States and edge cases	• Empty state from PRD F7 • Start date in the past: validation error • End date before start date: validation error

Reports (reports.index, GET /reports?month=YYYY-MM)

Access	Authenticated
Purpose	Analyse a month and export data.
Reached from	Sidebar; Dashboard link.
Main content	• Month switcher • Summary: income, expenses, net, savings rate • Category breakdown table (category, spent, share, budget, variance) next to the donut • 6-month trend bar chart with a data table beneath • Top 5 largest expenses • Export form: from, to, Download CSV
Primary and secondary actions	Primary: Download CSV. Secondary: month switcher.
After each action	Export returns a file download; validation errors return to the page with messages.
States and edge cases	• No data: "No data for this month." • Chart failure: retry button • Export range over 5 years: validation error

Settings (settings.edit, GET /settings)

Access	Authenticated
Purpose	Manage profile, password and account.
Reached from	Sidebar user area.
Main content	• Profile form: name, email, currency (with note that amounts are not converted) • Password form: current, new, confirm • Danger zone: delete account with password field
Primary and secondary actions	Primary: Save (each form has its own button). Secondary: Delete account (confirm dialog).
After each action	Each form redirects back to Settings with a message. Delete signs out and redirects to `/login` with "Your account and data were deleted."
States and edge cases	• Wrong current password error • Email already taken error

Error pages (n/a, 404, 419, 429, 500, 503)

Access	Any
Purpose	Explain what happened and offer a way out.
Reached from	Any failing request.
Main content	Title, one-sentence explanation, button to Dashboard (or Login if signed out).
Primary and secondary actions	Primary: Go to dashboard or sign in.
After each action	419: redirect to `/login` with "Your session expired. Please sign in again." Other codes render the branded page.
States and edge cases	• Never show stack traces in production

6. Primary User Journey (happy path)

1. Visit `/` and be redirected to `/login`. Choose Create an account.
2. Register with name, email, password and currency. Arrive on the Dashboard with the empty state.
3. Open Budgets, enter a limit for Groceries and Dining Out, Save.
4. Press Add transaction, choose Expense, Groceries, amount, date, Save.
5. See "Saved. Groceries: {spent} of {limit} used in {Mon YYYY} ({pct}%)." on the Transactions list.
6. Return to the Dashboard and see updated cards, donut and budget progress.

7. Secondary Journeys

- **Automate salary and rent**: Recurring, Add recurring transaction, save; or Process due now to see it appear immediately.
- **Monthly review**: Reports, switch month, read breakdown and trend, Download CSV.
- **Plan next month**: Budgets, switch to next month, Copy from previous month, adjust limits, Save.
- **Retire a category**: Categories, Archive; it disappears from pickers but stays in history.
- **Forgot password**: Login, Forgot password, request link, open email, set new password, sign in.
- **Leave the service**: Settings, Delete account, enter password, confirm.

8. Decision Points

User action	Condition	Result	Destination
Open `/`	Signed in?	Yes: Dashboard. No: Login	`dashboard` or `login`
Submit login	Credentials valid and not throttled	Valid: session started	Intended URL or `dashboard`
Save expense	Budget exists for category and month	Yes: status message with percent. No: plain "Transaction saved."	`transactions.index`
Save budgets	Limit blank for a category	Existing budget deleted; none created	`budgets.index`
Delete category	Has transactions or recurring rules	Yes: blocked with message. No: delete	`categories.index`
Change category type	Category is used	Used: type locked. Unused: allowed	`categories.index`
Pause rule	Rule active	Sets inactive; no new transactions	`recurring.index`
Resume rule	Rule paused	Next run set to first occurrence on or after today	`recurring.index`
Process due now	Any due rules	Creates N transactions or reports nothing due	`recurring.index`
Export CSV	Valid range	Valid: file download. Invalid: validation error	Download or `reports.index`
Open another user's record URL	Not the owner	Not found	404 page

9. Edge Cases and Recovery

Situation	Behaviour
Invalid input	Redirect back with old input (not passwords) and inline errors under each field; focus moves to the error summary at the top of the form.
Failed request (500)	Branded 500 page with a button back to the Dashboard. Details only in logs.
Lost connection (chart fetch)	Chart card shows "Could not load this chart." with a Retry button; page content stays usable.
Missing permission	404 page (never reveal that the record exists).
Expired session or CSRF token (419)	Redirect to `/login` with "Your session expired. Please sign in again."
Cancelled action	Confirm dialog Cancel closes the dialog and returns focus to the button that opened it. Nothing changes.
Double submit	Submit guard disables the button after the first submit.

10. Navigation Rules

Global navigation	Sidebar (desktop) or drawer (mobile) with: Dashboard, Transactions, Budgets, Recurring, Reports, Categories, Settings. Header contains page title and an "Add transaction" button on every authenticated screen. User name and Sign out at the bottom of the sidebar.
Back behaviour	Use normal browser history. After create, update or delete, the app redirects (Post/Redirect/Get). Cancel links go to the list the user came from.
Protected routes	Everything except login, register, forgot and reset password requires the `auth` middleware. Guest-only routes use `guest`.
Deep links	Intended URL after login. Query parameters `month`, `from`, `to`, `type`, `category_id`, `q`, `page` are shareable between the user's own sessions.
Active state	The current sidebar item has `aria-current="page"`.

11. Flowchart (Mermaid)

```
flowchart TD
  A[Visit /] --> B{Signed in?}
  B -- No --> C[Login]
  B -- Yes --> D[Dashboard]
  C -- Create account --> R[Register]
  R -- Success --> D
  C -- Forgot password --> F[Forgot password] --> G[Reset password] --> C
  C -- Valid login --> D
  D --> T[Transactions]
  D --> BU[Budgets]
  D --> RP[Reports]
  D --> RE[Recurring]
  D --> CA[Categories]
  D --> ST[Settings]
  D -- Add transaction --> TF[Transaction form]
  T -- Add or Edit --> TF
  TF -- Save --> T
  TF -- Cancel --> T
  T -- Delete --> DC{Confirm?}
  DC -- Yes --> T
  DC -- No --> T
  BU -- Save or Copy --> BU
  RE -- Add or Edit --> RF[Recurring form] --> RE
  RE -- Process due now --> RE
  RP -- Download CSV --> CSV[(CSV file)]
  ST -- Delete account --> C
  D -- Sign out --> C
```

Document 4: UI/UX Design Brief

One coherent visual system. Implement it as CSS custom properties in `resources/css/tokens.css` and reuse them everywhere. Never hard-code a colour, size or spacing value outside the token file.

1. Experience Goal

Desired feeling	Calm control. Like a well-kept paper ledger: orderly, trustworthy and quiet. The user should feel informed, never judged or alarmed.
Three visual adjectives	Calm, precise, warm.
The design must avoid	• Neon fintech gradients, glassmorphism and glow effects • Identical rounded cards stacked everywhere • Red used for everything (red only for genuine over-budget or destructive actions) • Emoji as icons, stock illustrations, gamification, confetti • Crypto-dashboard styling and dense dark panels

2. Users and Context

Primary user	Student or early-career professional logging expenses one-handed on a phone and reviewing on a laptop.
Device and environment	Phone (360 to 430 px wide) for quick entry, laptop (1280 px and wider) for review. Often used outdoors or in motion: large tap targets and high contrast.
Accessibility needs	WCAG 2.2 AA, keyboard-only use, screen readers, reduced motion, text zoom to 200%.

3. Visual Direction

Colour palette and roles (light theme only in v1)

Token	Hex	Role
`--color-bg`	#F6F4EE	Page background (warm paper)
`--color-surface`	#FFFFFF	Panels, inputs, dialogs, summary tiles
`--color-surface-alt`	#EFEBE0	Hover rows, table zebra, progress track, code blocks
`--color-border`	#DDD7C8	Hairlines, ledger rules, card borders
`--color-border-strong`	#B9B2A0	Input borders
`--color-text`	#1F2A28	Body text and headings
`--color-text-muted`	#5B6764	Secondary text, labels (4.5:1 or better on bg)
`--color-primary`	#1F5F4D	Primary buttons, links, active nav, focus accents
`--color-primary-hover`	#17493B	Primary hover and pressed
`--color-on-primary`	#FFFFFF	Text on primary
`--color-income`	#2E7D4F	Income amounts
`--color-expense`	#B4432F	Expense amounts and over-budget status
`--color-warning`	#B7791F	Nearing-limit bar fill and icon only (never text on white)
`--color-warning-text`	#7A4E0C	Text for warning messages
`--color-danger`	#A63A2A	Destructive buttons and error text
`--color-info`	#2F5F8A	Focus ring and informational notices
`--color-focus`	#2F5F8A	2 px focus outline colour

Category colours (8 presets, stored as hex in `categories.color`): `#1F5F4D` pine, `#B4432F` brick, `#2F5F8A` slate blue, `#B7791F` ochre, `#6B4E8F` plum, `#3D8B8B` teal, `#8A5A44` umber, `#6F7D3A` olive. Chart slices use these colours and are always accompanied by a legend with values.

Typography

Use	Family	Size / line height	Weight
Page title (h1)	Fraunces Variable, serif fallback `Georgia, serif`	28 px / 34 px (36 px / 42 px from 1024 px)	600
Section title (h2)	Fraunces Variable	22 px / 28 px	600
Subsection (h3)	Inter Variable	16 px / 24 px	600
Body	Inter Variable, `system-ui, sans-serif` fallback	16 px / 24 px	400
Small and labels	Inter Variable	14 px / 20 px	500
Caption and table headers	Inter Variable	12 px / 16 px, uppercase with 0.04em letter spacing	600
Large figures (summary tiles)	Fraunces Variable	28 px / 34 px	600

All numbers and currency use `font-variant-numeric: tabular-nums` so columns align. Use `rem` units; the root font size stays 100% (16 px).

Icons, imagery and surfaces

- Icons: simple inline SVG, 20 px, 1.75 px stroke, rounded caps, drawn in the style of the Lucide set. Provide an `<x-icon name="..."/>` Blade component. Icons never carry meaning alone; pair with text or `aria-label`.
- Imagery: none. Empty states use a small line drawing made of CSS or SVG rules (a ledger line motif) plus text.
- Surfaces: flat. 1 px borders using `--color-border`; corner radius 6 px for inputs and buttons, 10 px for summary tiles and dialogs. Lists and tables are ruled with hairlines like a ledger, not wrapped in repeated cards.
- Shadows: only on dialogs and drawers: `0 8px 24px rgba(31, 42, 40, 0.12)`.

4. Layout System

Content width	Maximum 1120 px, centred, with 16 px side padding on mobile and 32 px from 1024 px.
Shell	Desktop (1024 px and wider): fixed sidebar 248 px plus content area. Mobile: sticky top bar (menu button, title, Add button) and a slide-in drawer.
Grid	12 columns with 24 px gutters on desktop; 4 columns with 16 px gutters on mobile. Summary tiles use 4 columns on desktop, 2 on tablet, 1 on phones (or 2 across if space allows).
Spacing scale (4 px base)	`--space-1` 4, `--space-2` 8, `--space-3` 12, `--space-4` 16, `--space-5` 24, `--space-6` 32, `--space-7` 48, `--space-8` 64 (px).
Section rhythm	32 px between page sections on mobile, 48 px on desktop. 16 px between a heading and its content.
Breakpoints (mobile first, `min-width`)	640 px (tablet), 1024 px (desktop). No `max-width` queries.

5. Component Language

States required for every interactive component: default, hover, focus-visible, active, disabled, loading and error where relevant.

Component	Specification
Buttons	• Variants: primary (filled `--color-primary`), secondary (surface with border), ghost (text only), danger (filled `--color-danger`) • Height 40 px desktop, 44 px minimum on touch screens; horizontal padding 16 px; font 14 px, weight 600 • Hover: darker fill (primary-hover) or `--color-surface-alt` background. Active: 1 px translateY. Disabled: 50% opacity, `cursor: not-allowed` • Loading: label replaced by "Saving..." (or similar), `aria-busy="true"`, button disabled by submit-guard
Inputs	• Visible label above the field (never placeholder-only). 44 px tall on touch, 40 px on desktop. Font size at least 16 px on mobile to prevent iOS zoom • Border 1 px `--color-border-strong`; focus: 2 px `--color-focus` outline with 2 px offset plus primary border • Error: border `--color-danger`, message below with an icon and `aria-describedby`, `aria-invalid="true"` • Amount field: `inputmode="decimal"`, right-aligned digits, currency code shown as suffix text
Navigation	• Sidebar items: icon + label, 44 px high. Active: 3 px left bar in primary, `--color-surface-alt` background, `aria-current="page"` • Drawer (mobile): native-feeling slide from left, backdrop `rgba(31,42,40,0.4)`, closes on Escape, backdrop click and link click, focus returns to the menu button
Summary tiles	• Surface, 1 px border, radius 10 px, padding 16 px (24 px desktop). Label in caption style, value in Fraunces 28 px. A small coloured left rule (income, expense, neutral, budget) 3 px wide • Tiles are not links; no hover state
Dialogs	• Use the native `<dialog>` element opened with `showModal()`. Title, one sentence of text, Cancel (secondary) and Confirm (danger) buttons. Escape closes. Focus returns to the opener. Backdrop `rgba(31,42,40,0.4)`
Tables and lists	• Ledger style: no outer card; header row in caption style; 1 px bottom rule per row; row hover `--color-surface-alt` • Amounts right-aligned, tabular figures. Expenses show a leading minus sign and `--color-expense`; income shows a plus sign and `--color-income`. Add visually hidden text "Expense" or "Income" for screen readers • Mobile: rows become stacked items (description and category left, amount right, date beneath)
Feedback and status	• Flash banners at the top of main content: success (primary tint `#E5F0EC`), error (`#F7E4DF`), warning (`#F8EBD2`), each with icon, text, dismiss button; `role="status"` (success) or `role="alert"` (error) • Budget progress bar: 8 px tall track `--color-surface-alt`, fill by status (On track: primary, Nearing limit: warning, Over budget: expense) plus status text beside the figures • Empty states: short heading, one sentence, one primary button • Spinner for chart loading: 20 px circle, 600 ms rotation (static "Loading chart..." text under reduced motion)
Badges	• "Recurring", "Archived", "Paused", "Ended": 12 px caption text in a bordered pill, neutral colours

6. Screen Direction

Screen	Hierarchy and composition	Primary action	Responsive changes
Dashboard	Month switcher and title first; four summary tiles; then two columns: donut with legend (left) and budgets list (right); recent transactions as a ruled list below.	Add transaction in the header	Columns stack; tiles 2 across on phones.
Transactions	Filter bar in a single row; totals strip; ruled list; pagination.	Add transaction in the header	Filters collapse into a "Filters" disclosure on phones; rows stack.
Add and Edit transaction	Single column, max 480 px. Order: type toggle, category, amount, date, description. The amount is visually the largest field.	Save button at the bottom (sticky on phones)	Buttons full width on phones.
Budgets	Totals strip, then one ruled row per category with input and progress bar.	Save budgets (sticky bar on phones)	Rows stack: name and status above, input and numbers below.
Reports	Summary strip, donut plus breakdown table, trend chart with table, top expenses, export block.	Download CSV	Single column; tables scroll horizontally inside their container.
Recurring	Ruled list with status badges; Process due now as a secondary button.	Add recurring transaction	Rows stack.
Categories	Two sections (Expense, Income) with colour dots.	Add category	Rows stack.
Auth screens	Centred form 400 px wide on a plain background with the wordmark above. No hero images.	Submit button	Full width on phones.
Settings	Three stacked sections separated by hairlines; danger zone has a danger-coloured top rule.	Section Save buttons	Single column.

7. Interaction and Motion

- Purposeful transitions only: colour and background changes 150 ms ease-out; drawer slide 200 ms; progress bar width from 0 on load 300 ms; dialog fade 150 ms.
- Feedback moments: flash banner on every successful write; button loading label; inline field errors.
- Reduced motion: under `@media (prefers-reduced-motion: reduce)` set all transition and animation durations to 0.01 ms and disable Chart.js animation (`Chart.defaults.animation = false`).
- No parallax, bounce, confetti or looping animations.

8. Accessibility

- **Contrast**: text at least 4.5:1, large text and UI components at least 3:1. Never use `--color-warning` for text on light backgrounds.
- **Keyboard**: every control reachable in a logical order; "Skip to main content" link as the first focusable element; drawer and dialogs trap focus correctly and restore it.
- **Focus states**: 2 px outline `--color-focus`, 2 px offset, never removed without replacement (use `:focus-visible`).
- **Tap targets**: at least 44 by 44 px on touch screens.
- **Text sizing**: rem units; layout works at 200% zoom and 320 px width without horizontal page scroll.
- **Semantics**: `lang="en"`; landmarks (`header`, `nav`, `main`); unique `<title>` such as "Budgets - PocketLedger"; tables use `<th scope>`; icons decorative with `aria-hidden="true"` unless they carry meaning; flash messages use `role="status"` or `role="alert"`.
- **Charts**: each canvas has `role="img"` and an `aria-label` summary, plus the legend or table with the same numbers. Colour is never the only differentiator.
- **Status**: budget status is conveyed with words ("On track", "Nearing limit", "Over budget") as well as colour.

9. Formatting Rules for Money and Dates

- Money: formatted by `CurrencyFormatter` (PHP) and `formatMoney` (TypeScript) using the user's currency. INR uses Indian grouping (for example 12,34,567.00).
- Dates: "12 Oct 2026" in lists, "October 2026" for months, `<input type="date">` and `<input type="month">` for inputs. Always include a visible text label.
- Percent: whole numbers (70%) except savings rate with one decimal.
- Tone of copy: plain, kind, specific. Say "Over budget by 1,200.00", not "You failed your budget".

10. Consistency Rules

Always use

- Tokens from `tokens.css` for every colour, spacing and radius
- The Blade components `x-button`, `x-field`, `x-flash`, `x-icon`, `x-stat`, `x-budget-bar`, `x-empty-state`, `x-month-switcher`, `x-amount`
- Ledger-style ruled lists for data, summary tiles only for the four headline figures
- Words plus colour for status
- Post/Redirect/Get with a flash message after every write
- Sentence case for buttons and headings ("Add transaction")

Never use

- Gradients, glassmorphism, blur backdrops, neon accents
- Emoji or icon-only buttons without an accessible name
- Placeholder text as the only label
- Inline styles or `!important`
- Colour alone to indicate status
- Third-party CSS frameworks or UI kits
- Autoplaying or looping motion

Document 5: Backend Schema

MySQL 8, InnoDB, `utf8mb4` with `utf8mb4_unicode_ci`. Names below are exact. Every table, column and relationship supports a named PRD requirement (see the traceability list at the end).

1. Data Overview

PocketLedger stores five business tables owned by a user: **categories** (how money is grouped), **transactions** (the ledger), **budgets** (monthly limits per expense category) and **recurring_transactions** (rules that generate transactions), plus the **users** table and Laravel's standard infrastructure tables for sessions, cache and password resets. Every business row carries `user_id` and is only ever accessed by its owner.

Database type	MySQL 8.0+, InnoDB, `utf8mb4`
Main data domains	Identity, categorisation, ledger, planning, automation
Ownership model	Single owner per row via `user_id`. No sharing.
Money columns	`DECIMAL(12,2)`. Never FLOAT or DOUBLE.

2. Authentication and Authorisation

User identity	Unique email address.
Sign-in methods	Email and password only.
Roles	None. Every user is an account owner.
Permissions	Owner-only access to all rows through Policies and scoped queries.
Session handling	Database sessions (`sessions` table), 120 minute lifetime, regenerate on login, invalidate on logout. Remember-me token in `users.remember_token`.

3. Tables

Create migrations in this order so foreign keys resolve: Laravel defaults (users, password_reset_tokens, sessions, cache, jobs), then `categories`, `recurring_transactions`, `transactions`, `budgets`.

Table `users`

Account owners. Supports F1. Start from Laravel's default users migration and apply the changes below.

Column	Type	Null	Default	Notes
id	BIGINT UNSIGNED	No	auto	Primary key
name	VARCHAR(100)	No		
email	VARCHAR(255)	No		UNIQUE
password	VARCHAR(255)	No		Hashed via the `hashed` cast
currency_code	CHAR(3)	No	`INR`	One of INR, USD, EUR, GBP, AED, SGD, AUD, CAD
remember_token	VARCHAR(100)	Yes	NULL	
created_at, updated_at	TIMESTAMP	Yes	NULL	

- Remove `email_verified_at` from the default migration (email verification is out of scope).

Table `categories`

Income and expense categories. Supports F2 and every other feature that classifies money.

Column	Type	Null	Default	Notes
id	BIGINT UNSIGNED	No	auto	Primary key
user_id	BIGINT UNSIGNED	No		FK users(id) ON DELETE CASCADE
name	VARCHAR(60)	No		Trimmed, 1 to 60 characters
type	ENUM('income','expense')	No		PHP enum `CategoryType`
color	CHAR(7)	No	`#1F5F4D`	Hex from the 8 preset colours
is_archived	TINYINT(1)	No	0	Archived categories stay in history
created_at, updated_at	TIMESTAMP	Yes	NULL	

- UNIQUE (user_id, type, name). The collation makes it case-insensitive.
- INDEX (user_id, is_archived).

Table `recurring_transactions`

Rules that generate transactions. Supports F7.

Column	Type	Null	Default	Notes
id	BIGINT UNSIGNED	No	auto	Primary key
user_id	BIGINT UNSIGNED	No		FK users(id) ON DELETE CASCADE
category_id	BIGINT UNSIGNED	No		FK categories(id) ON DELETE RESTRICT
amount	DECIMAL(12,2)	No		CHECK (amount > 0)
description	VARCHAR(255)	Yes	NULL	
frequency	ENUM('daily','weekly','monthly','yearly')	No		PHP enum `Frequency`
start_date	DATE	No		Anchor for every occurrence
end_date	DATE	Yes	NULL	CHECK (end_date IS NULL OR end_date >= start_date)
next_run_date	DATE	Yes	NULL	Next occurrence to create; NULL when the rule has ended
last_run_date	DATE	Yes	NULL	Most recent occurrence created
runs_count	INT UNSIGNED	No	0	Number of occurrences created so far (index n for the calculator)
is_active	TINYINT(1)	No	1	Paused when 0
created_at, updated_at	TIMESTAMP	Yes	NULL	

- INDEX (is_active, next_run_date) for the scheduler.
- INDEX (user_id).
- Status shown in the UI: Ended when `next_run_date` is NULL, Paused when `is_active` is 0, otherwise Active.

Table `transactions`

The ledger. Supports F3, F4 (spent amounts), F5, F6 and F7 (generated rows).

Column	Type	Null	Default	Notes
id	BIGINT UNSIGNED	No	auto	Primary key
user_id	BIGINT UNSIGNED	No		FK users(id) ON DELETE CASCADE
category_id	BIGINT UNSIGNED	No		FK categories(id) ON DELETE RESTRICT

Column	Type	Null	Default	Notes
amount	DECIMAL(12,2)	No		CHECK (amount > 0); direction comes from the category type
transaction_date	DATE	No		Calendar date
description	VARCHAR(255)	Yes	NULL	
recurring_transaction_id	BIGINT UNSIGNED	Yes	NULL	FK recurring_transactions(id) ON DELETE SET NULL
created_at, updated_at	TIMESTAMP	Yes	NULL	

- INDEX (user_id, transaction_date) for lists and month totals.
- INDEX (user_id, category_id, transaction_date) for category filters and breakdowns.
- UNIQUE (recurring_transaction_id, transaction_date) makes recurring generation idempotent. MySQL allows many NULLs in a unique index, so manual transactions are unaffected.
- Add the CHECK constraint with `DB::statement('ALTER TABLE transactions ADD CONSTRAINT chk_transactions_amount_positive CHECK (amount > 0)')` in the migration.

Table `budgets`

Monthly spending limit per expense category. Supports F4, F5 and F6.

Column	Type	Null	Default	Notes
id	BIGINT UNSIGNED	No	auto	Primary key
user_id	BIGINT UNSIGNED	No		FK users(id) ON DELETE CASCADE
category_id	BIGINT UNSIGNED	No		FK categories(id) ON DELETE CASCADE
month	DATE	No		Always the first day of the month (for example 2026-10-01)
amount_limit	DECIMAL(12,2)	No		CHECK (amount_limit > 0)
created_at, updated_at	TIMESTAMP	Yes	NULL	

- UNIQUE (user_id, category_id, month).
- INDEX (user_id, month).
- Only expense categories of the same user may be referenced (enforced in validation).

Laravel infrastructure tables (keep Laravel defaults)

- `password_reset_tokens` (email primary key, token, created_at)
- `sessions` (id, user_id nullable index, ip_address, user_agent, payload, last_activity index)
- `cache` and `cache_locks` (used by the database cache store and rate limiter)
- `jobs`, `job_batches`, `failed_jobs` may remain but are unused (queue is `sync`)

4. Relationships and Delete Behaviour

Relationship	Type	On delete
User has many Categories	One-to-many	CASCADE (account deletion; see explicit order in section 8)
User has many Transactions	One-to-many	CASCADE
User has many Budgets	One-to-many	CASCADE
User has many RecurringTransactions	One-to-many	CASCADE
Category has many Transactions	One-to-many	RESTRICT (a used category can only be archived)
Category has many RecurringTransactions	One-to-many	RESTRICT
Category has many Budgets	One-to-many	CASCADE (budgets go with an unused category)
RecurringTransaction has many Transactions	One-to-many	SET NULL (history kept when a rule is deleted)

There are no one-to-one or many-to-many relationships in version one.

5. Access Rules

Table	Create	Read	Update	Delete
users	Anyone (registration)	Self	Self	Self (with password)
categories	Owner	Owner	Owner	Owner, only if unused
transactions	Owner; scheduler for owner's rules	Owner	Owner	Owner
budgets	Owner	Owner	Owner	Owner
recurring_transactions	Owner	Owner	Owner	Owner

Enforcement: Policies (`CategoryPolicy`, `TransactionPolicy`, `RecurringTransactionPolicy`) and relationship-scoped queries (`$user->transactions()`). A request for a record the user does not own results in HTTP 404.

6. Core Data Operations (SQL contracts)

Implement these with the Laravel query builder, binding parameters. `:start` is the first day of the month, `:next_start` is the first day of the next month. All comply with `ONLY_FULL_GROUP_BY`.

Month totals by type (Dashboard cards, Reports summary)

```
SELECT c.type, SUM(t.amount) AS total
FROM transactions t
JOIN categories c ON c.id = t.category_id
WHERE t.user_id = :user
  AND t.transaction_date >= :start AND t.transaction_date < :next_start
GROUP BY c.type;
```

Expense by category with budget (donut, Budgets page, breakdown table)

```
SELECT c.id, c.name, c.color, b.amount_limit, COALESCE(SUM(t.amount), 0) AS spent
FROM categories c
LEFT JOIN transactions t
       ON t.category_id = c.id AND t.user_id = c.user_id
      AND t.transaction_date >= :start AND t.transaction_date < :next_start
LEFT JOIN budgets b
       ON b.category_id = c.id AND b.user_id = c.user_id AND b.month = :start
WHERE c.user_id = :user AND c.type = 'expense'
  AND (c.is_archived = 0 OR t.id IS NOT NULL OR b.id IS NOT NULL)
GROUP BY c.id, c.name, c.color, b.amount_limit
ORDER BY spent DESC, c.name ASC;
```

Six-month trend (Reports)

```
SELECT DATE_FORMAT(t.transaction_date, '%Y-%m') AS ym, c.type, SUM(t.amount) AS total
FROM transactions t
JOIN categories c ON c.id = t.category_id
WHERE t.user_id = :user
  AND t.transaction_date >= :range_start AND t.transaction_date < :range_end
GROUP BY ym, c.type
ORDER BY ym;
```

Fill months with no rows with zero in PHP so the response always contains exactly 6 months, oldest first.

Transactions list with filtered totals

- List: `Transaction::query()->whereBelongsTo($user)->with('category')` plus filters, ordered by `transaction_date DESC, id DESC`, `paginate(20)->withQueryString()`.
- Search: `description LIKE %term%` after escaping `%`, `_` and `\` in the term.
- Totals: a separate aggregate query with the same filters joined to categories and grouped by `c.type` (not computed from the current page).

Copy previous month budgets

```
INSERT IGNORE INTO budgets (user_id, category_id, month, amount_limit, created_at, updated_at)
SELECT b.user_id, b.category_id, :this_month, b.amount_limit, NOW(), NOW()
FROM budgets b
JOIN categories c ON c.id = b.category_id
WHERE b.user_id = :user AND b.month = :prev_month AND c.is_archived = 0;
```

Recurring processing (pseudocode)

```
today = Carbon::today()   // app timezone
RecurringTransaction::where('is_active', 1)->whereNotNull('next_run_date')
    ->where('next_run_date', '<=', today)->chunkById(200, function ($rules) {
  foreach ($rules as $rule) {
    DB::transaction(function () use ($rule, $today) {
      $r = RecurringTransaction::lockForUpdate()->find($rule->id);
      $guard = 0;
      while ($r->next_run_date && $r->next_run_date <= $today && $guard++ < 400) {
        if ($r->end_date && $r->next_run_date > $r->end_date) break;
        Transaction::insertOrIgnore([... user_id, category_id, amount, description,
            'transaction_date' => $r->next_run_date, 'recurring_transaction_id' => $r->id ...]);
        $r->runs_count++;  $r->last_run_date = $r->next_run_date;
        $r->next_run_date = RecurrenceCalculator::occurrence($r->start_date,
                                                             $r->frequency, $r->runs_count);
      }
      if ($r->end_date && $r->next_run_date && $r->next_run_date > $r->end_date) {
        $r->next_run_date = null;   // ended
      }
      $r->save();
    });
  }
});
```

RecurrenceCalculator::occurrence(start, frequency, n)

- daily: `start + n days`; weekly: `start + n weeks`
- monthly: `start->copy()->addMonthsNoOverflow(n)` computed from the start date every time (so day 31 returns after short months)
- yearly: `start->copy()->addYearsNoOverflow(n)` (29 Feb becomes 28 Feb in non-leap years)
- Resume after pause: set `runs_count` and `next_run_date` to the first occurrence on or after today by incrementing n until the date is not before today.

7. File Storage

Not used. No upload tables, buckets or folders.

8. Data Integrity and Security

Validation	Every rule in Appendix B is enforced in Form Requests and backed by constraints (FK, UNIQUE, CHECK, NOT NULL).
Transaction boundaries	• Registration: create user and default categories in one `DB::transaction` • Budgets bulk save: upsert and delete in one transaction • Recurring processing: one transaction per rule • Account deletion: one transaction
Account deletion order	Inside one `DB::transaction` delete rows of the user in this order: transactions, recurring_transactions, budgets, categories, then the user (sessions rows with the user id are also deleted). This avoids RESTRICT conflicts.
Sensitive data	Only email, name and password hash are personal data. Never log amounts or request bodies. No card, bank or government ID numbers are ever stored.
Audit requirements	None beyond `created_at` and `updated_at`.

9. Migrations, Factories and Seed Data

- Migrations in the order of section 3. Each migration has a working `down()`.
- Factories for `User`, `Category`, `Transaction`, `Budget`, `RecurringTransaction` (used by tests and the demo seeder).
- `SeedDefaultCategories` action creates the defaults from Appendix D for a user (used at registration and by the demo seeder).
- `DemoSeeder` (run explicitly with `php artisan db:seed --class=DemoSeeder`; also run automatically when `APP_ENV=local`) creates user `demo@pocketledger.test` with password `password`, currency INR, default categories, about 12 months of deterministic realistic transactions (seeded random generator), budgets for the current and previous month, and two recurring rules (salary monthly, rent monthly). Never create this user when `APP_ENV=production` unless the command is run explicitly.

10. Risks and Open Questions

- MySQL CHECK constraints require 8.0.16 or newer. Document this in the README.
- Case-insensitive uniqueness relies on the default collation; do not change it.
- Month boundaries use half-open ranges (`>= start AND < next_start`) to avoid time-of-day problems.
- No unresolved questions.

Entity-Relationship Diagram (Mermaid)

```
erDiagram
  USERS ||--o{ CATEGORIES : owns
  USERS ||--o{ TRANSACTIONS : owns
  USERS ||--o{ BUDGETS : owns
  USERS ||--o{ RECURRING_TRANSACTIONS : owns
  CATEGORIES ||--o{ TRANSACTIONS : classifies
  CATEGORIES ||--o{ BUDGETS : limits
  CATEGORIES ||--o{ RECURRING_TRANSACTIONS : classifies
  RECURRING_TRANSACTIONS ||--o{ TRANSACTIONS : generates
  USERS {
    bigint id PK
    string name
    string email UK
    string currency_code
  }
  CATEGORIES {
    bigint id PK
    bigint user_id FK
    string name
    enum type
    string color
    bool is_archived
  }
  TRANSACTIONS {
    bigint id PK
    bigint user_id FK
    bigint category_id FK
    decimal amount
    date transaction_date
  }
  BUDGETS {
    bigint id PK
    bigint user_id FK
    bigint category_id FK
    date month
    decimal amount_limit
  }
  RECURRING_TRANSACTIONS {
    bigint id PK
    bigint user_id FK
    bigint category_id FK
    enum frequency
    date next_run_date
  }
```

Traceability: Table to PRD Feature

Table	Features and requirements supported
users	F1 (AUTH-01 to AUTH-05), F8 (OWN-01), currency display for all money
categories	F2 (CAT-01 to CAT-04), used by F3, F4, F5, F6, F7
transactions	F3 (TXN-01 to TXN-05), F5 (DASH-01 to DASH-03), F6 (RPT-01 to RPT-05), F7 (REC-03)
budgets	F4 (BUD-01 to BUD-04), TXN-05, DASH-01, DASH-03, RPT-02
recurring_transactions	F7 (REC-01 to REC-06)
sessions, cache, password_reset_tokens	AUTH-02, AUTH-04 and rate limiting

Document 6: Implementation Plan

Ordered, verifiable build sequence for the AI coding agent. Complete phases in order. Do not start a phase while the previous phase's checks fail. Commit after every phase.

1. Current Project State

Existing code	None assumed. Start from an empty directory. If a Laravel skeleton already exists, reuse it and remove what the TRD excludes.
Reusable parts	Laravel framework defaults (users migration, session and cache tables, authentication guard).
Missing foundations	Everything else: tooling, design system, schema, features, tests, CI, docs.
Risks	Laravel's current skeleton ships with Tailwind and Axios; they must be removed. MySQL must be running before migrations and tests.

2. Build Principles

- Preserve the approved scope (PRD section 5).
- Work in small verifiable phases and commit each as `Phase N: <title>`.
- Test after every phase using the listed commands.
- Do not begin later phases while required checks fail.
- Record every unspecified choice in `docs/DECISIONS.md`.
- Keep controllers thin; business rules live in Actions and Services.
- Write tests with the feature, not afterwards.

3. Ordered Phases

Phase 0: Project foundation and tooling

Goal	A runnable Laravel app on MySQL with Vite, TypeScript, tests, linting and CI in place.
Requirements covered	TRD sections 2, 7, 8
Work to do	• Create the project (Appendix A commands); remove Tailwind, Axios, `resources/js` and the default `welcome` view styling • Configure `.env`, `.env.example`, `APP_TIMEZONE=Asia/Kolkata`, database drivers (session, cache) as in Appendix A • Create databases `pocketledger` and `pocketledger_test`; set testing DB in `phpunit.xml` • Install Pest, Pint, Larastan; configure `phpstan.neon` (level 6) and `pint.json` (Laravel preset) • Install TypeScript, ESLint, Vitest; add `tsconfig.json`, `vite.config.ts`, `eslint.config.js`, `vitest.config.ts`, npm scripts • Add `.github/workflows/ci.yml`, `docs/DECISIONS.md`, `docs/SPEC.pdf`, minimal `README.md` • Add one trivial route and Pest test, one trivial Vitest test to prove the toolchain
Tests and verification	• `php artisan migrate:fresh` • `npm run typecheck && npm run lint && npm run test:ts && npm run build` • `./vendor/bin/pint --test && ./vendor/bin/phpstan analyse && ./vendor/bin/pest`
Completion criteria	All commands above succeed on a clean checkout; CI file present.
Dependencies	None.
Risks and rollback	If MySQL is unreachable, stop and tell the human. Do not fall back to SQLite.

Phase 1: Design system and layout

Goal	Reusable CSS tokens, base styles, layouts and Blade components exactly as in the UI/UX Brief, with the small TypeScript behaviours the shell needs.
Requirements covered	UI/UX Brief sections 3 to 10; TRD TypeScript modules (nav-drawer, flash, confirm-dialog, submit-guard)
Work to do	• `resources/css/`: `tokens.css`, `base.css` (reset, typography, focus), `layout.css` (shell, grid), `components/*.css`, `pages/*.css`, imported from `app.css` • Fonts imported from Fontsource in `ts/app.ts` • Blade layouts `layouts/app.blade.php` (sidebar, top bar, drawer, skip link, flash area) and `layouts/guest.blade.php` • Blade components: `x-button`, `x-field`, `x-flash`, `x-icon` (with the icons needed across the app), `x-stat`, `x-budget-bar`, `x-empty-state`, `x-month-switcher`, `x-amount` • Branded error views `errors/404`, `419`, `429`, `500`, `503` • TS modules: `nav-drawer`, `flash`, `confirm-dialog`, `submit-guard`, `lib/dom` • A local-only route `/_styleguide` (registered only when `APP_ENV=local`) showing all components and states
Tests and verification	• Manual check of `/_styleguide` at 360, 768 and 1280 px; keyboard pass • `npm run typecheck && npm run lint && npm run build` • Feature test that `/_styleguide` returns 404 outside local
Completion criteria	Layouts and components render correctly in all states; contrast and focus rules met; reduced-motion media query present.
Dependencies	Phase 0.
Risks and rollback	Keep components generic; avoid page-specific styles in component files.

Phase 2: Database schema, models and factories

Goal	The full schema from the Backend Schema with models, enums, relationships, casts and factories.
Requirements covered	Backend Schema sections 3, 4, 9
Work to do	• Migrations in the documented order including CHECK constraints and indexes • Models with `$fillable`, casts (`decimal:2`, dates, enums, booleans), relationships, query scopes (`active()`, `expense()`, `income()`) • Enums `CategoryType`, `Frequency`, `BudgetStatus` • Factories for every model • Policies registered for Category, Transaction, RecurringTransaction • `MonthRange` value object (start, next start, label, previous, next) and `CurrencyFormatter` service
Tests and verification	• `php artisan migrate:fresh` • Pest unit tests for `MonthRange`, `CurrencyFormatter` • Pest feature tests proving constraints: duplicate category name rejected, negative amount rejected by CHECK, duplicate recurring occurrence rejected
Completion criteria	Schema matches the Backend Schema exactly (names, types, indexes, FKs). `migrate:fresh` and `migrate:rollback` both work.
Dependencies	Phase 0.
Risks and rollback	Foreign key order: categories, recurring_transactions, transactions, budgets.

Phase 3: Authentication, settings and default categories

Goal	Users can register, sign in, reset password, manage settings and delete their account.
Requirements covered	F1; AUTH-01 to AUTH-05; F8
Work to do	• Controllers `RegisterController`, `LoginController`, `LogoutController`, `ForgotPasswordController`, `ResetPasswordController`, `SettingsController` with Form Requests • Actions `RegisterUser` (user plus defaults in one transaction), `SeedDefaultCategories`, `DeleteUserAccount` (explicit order) • Named rate limiters for login and password reset; intended-URL redirect; `guest` and `auth` middleware groups; `/` redirect • Views for auth screens and Settings (three forms); 419 handling redirecting to login with the session-expired message in `bootstrap/app.php` • Security headers middleware
Tests and verification	• Pest feature tests: register seeds categories and signs in; duplicate email; login success and failure; throttling; logout; password reset flow with `Notification::fake()`; settings updates; delete account removes all rows; guests redirected from protected routes • Manual: full auth journey in the browser
Completion criteria	All AUTH acceptance criteria pass; no protected route is reachable as a guest.
Dependencies	Phases 1 and 2.
Risks and rollback	Never repopulate password fields; test that responses do not contain the password.

Phase 4: Categories management

Goal	Users manage categories as specified in F2.
Requirements covered	F2; CAT-01 to CAT-04
Work to do	• `CategoryController` (index, create, store, edit, update, destroy, archive, restore) with Form Requests and `CategoryPolicy` • Views with Expense and Income sections, archived toggle, colour swatch picker • Delete guard and type-lock rules; transaction counts via `withCount`
Tests and verification	• Pest feature tests: CRUD, uniqueness per user and type, archive hides from pickers, cannot delete used category, cannot change type when used, foreign category returns 404
Completion criteria	F2 acceptance criteria pass.
Dependencies	Phases 2 and 3.
Risks and rollback	Keep pickers using a shared scope `Category::active()` to avoid divergence.

Phase 5: Transactions

Goal	Fast creation and complete management of transactions as in F3, including the budget status message.
Requirements covered	F3; TXN-01 to TXN-05
Work to do	• `TransactionController` with `StoreTransactionRequest` and `UpdateTransactionRequest`, `TransactionPolicy` • `BudgetStatusService` (percent, status, remaining) used here and later by Budgets and Dashboard • List query with filters, search escaping, totals query, pagination with query string, `return_to` handling (must start with `/transactions`) • Views: list (filters, totals strip, ruled rows, pagination), create and edit form with type toggle • TS: `transaction-form.ts` (category filtering by `data-type`), tests in Vitest for the filter function • Save-and-add-another on create
Tests and verification	• Pest feature tests for validation edges (0.00, 3 decimals, future date, archived or foreign category), filters, totals across pages, search escaping, budget message variants, `return_to` safety, constant query count on the list • `npm run test:ts`
Completion criteria	F3 acceptance criteria pass; list of 10,000 rows (factory) responds quickly with indexes used.
Dependencies	Phases 2, 3, 4.
Risks and rollback	Open redirect risk with `return_to`: validate strictly.

Phase 6: Budgets

Goal	Monthly budgets with status and copy from previous month (F4).
Requirements covered	F4; BUD-01 to BUD-04; TXN-05
Work to do	• `BudgetController` (index, update, copy) with `UpdateBudgetsRequest`, queries from Backend Schema section 6 • Budgets page view with rows, progress bars, totals; `x-budget-bar` status text • TS: `budget-form.ts` live total
Tests and verification	• Pest feature tests: blank removes budget, bulk save is transactional, copy never overwrites and skips archived categories, month bounds, status thresholds at 79.99%, 80%, 100%, 100.01%, only expense categories accepted
Completion criteria	F4 acceptance criteria pass.
Dependencies	Phase 5.
Risks and rollback	Use decimal-safe string math for percentages.

Phase 7: Dashboard, reports, charts and CSV export

Goal	Insight screens and data export (F5 and F6).
Requirements covered	F5, F6; DASH-01 to DASH-03; RPT-01 to RPT-05
Work to do	• `ReportService` implementing the SQL contracts • `DashboardController`, `ReportController`, `ReportDataController` (JSON per Appendix C) • Views for Dashboard and Reports with legends and data tables as text alternatives • TS: `chart-loader.ts`, `category-donut.ts`, `monthly-trend.ts` (Chart.js tree-shaken, reduced-motion aware, retry state) • CSV streaming with BOM and formula-injection protection
Tests and verification	• Pest feature tests with fixed fixtures asserting totals, shares, variance, trend zero-fill, top 5, savings rate with zero income, CSV header and injection prefixing, export range validation, JSON shapes • Vitest tests for chart data mapping • Manual: charts at 360 and 1280 px, reduced motion setting on
Completion criteria	F5 and F6 acceptance criteria pass; bundle under 150 KB gzipped (check `npm run build` output).
Dependencies	Phases 5 and 6.
Risks and rollback	Keep SQL compliant with `ONLY_FULL_GROUP_BY`; test against MySQL only.

Phase 8: Recurring transactions

Goal	Rules that create transactions automatically (F7).
Requirements covered	F7; REC-01 to REC-06
Work to do	• `RecurringTransactionController` and requests, `RecurringTransactionPolicy` • `RecurrenceCalculator`, `RecurringTransactionProcessor` • Artisan command `recurring:process` scheduled daily at 00:10 in `routes/console.php` with `withoutOverlapping()` • Pause and resume logic; "Process due now" action for the signed-in user • Views: list with status and next run, form
Tests and verification	• Pest unit tests for the calculator (daily, weekly, monthly including day 29, 30, 31 and leap years, yearly 29 Feb) • Feature tests: catch-up after missed days, idempotency on double run, end date stops, pause and resume, edit affects only future, delete keeps transactions, per-user process-due isolation • `php artisan recurring:process` runs without error; `php artisan schedule:list` shows the task
Completion criteria	F7 acceptance criteria pass.
Dependencies	Phase 5.
Risks and rollback	Time-dependent tests must use `Carbon::setTestNow()`.

Phase 9: Polish, demo data, documentation and deployment readiness

Goal	Production-quality finish.
Requirements covered	PRD sections 8 and 9; TRD sections 5 to 8
Work to do	• Empty, error, loading and success states reviewed on every screen against the App Flow • Responsive and accessibility pass (keyboard, focus, contrast, zoom, reduced motion) • `DemoSeeder` as specified; factories finalised • README: overview, screenshots placeholder, feature list, stack, local setup, test commands, demo login, deployment steps (TRD section 8), scheduler note • Final audit using the Final Consistency Check (section 7) • Remove dead code, unused routes and unused dependencies
Tests and verification	• Every command in section 6 (Final Verification) • Manual smoke script in section 6
Completion criteria	All checks pass; README lets a stranger run the project in under 10 minutes.
Dependencies	Phases 0 to 8.
Risks and rollback	Do not add features while polishing.

4. Suggested Sequence (summary)

Order	Phase	Produces
0	Foundation	Tooling, CI
1	Design system	CSS tokens, layouts, components
2	Schema and models	Migrations, models, factories
3	Auth and settings	Accounts, default categories
4	Categories	Category management
5	Transactions	Core ledger
6	Budgets	Plans and status
7	Dashboard and reports	Charts, export
8	Recurring	Automation
9	Polish and deploy	Quality, docs

5. Requirement Traceability

PRD feature	Screens	Data	Phase	Verification
F1 Auth and settings	Login, Register, Forgot, Reset, Settings	users, sessions, password_reset_tokens	3	Auth feature tests
F2 Categories	Categories (list, add, edit)	categories	4	Category feature tests
F3 Transactions	Transactions list, Add, Edit	transactions, categories, budgets	5	Transaction feature tests, Vitest
F4 Budgets	Budgets	budgets, transactions, categories	6	Budget feature tests
F5 Dashboard	Dashboard	transactions, budgets, categories	7	Dashboard tests, manual
F6 Reports and export	Reports	transactions, budgets, categories	7	Report and CSV tests
F7 Recurring	Recurring (list, add, edit)	recurring_transactions, transactions	8	Calculator and processor tests
F8 Privacy and ownership	All, error pages	user_id on every table	3 to 9	Ownership tests per controller

6. Final Verification

Automated (all must pass)

```
composer install && npm ci
php artisan migrate:fresh --seed          # includes DemoSeeder in local
./vendor/bin/pint --test
./vendor/bin/phpstan analyse --memory-limit=1G
./vendor/bin/pest
npm run typecheck && npm run lint && npm run test:ts
npm run build                              # production bundle succeeds, size under budget
php artisan route:list                     # no unused or unnamed required routes
php artisan schedule:list                  # recurring:process is scheduled
```

Manual smoke script (use a fresh browser profile)

1. Register a new account; confirm default categories and the Dashboard empty state.
2. Set budgets for two categories; add expenses; confirm the status message and progress bar thresholds (On track, Nearing limit, Over budget).
3. Filter and search transactions; confirm totals cover all pages; edit and delete with confirm dialog.
4. Create a monthly recurring rule starting today; use Process due now; confirm the transaction appears once and a second run creates nothing.
5. Open Reports for the month; confirm numbers match the Dashboard; download CSV and open it in a spreadsheet.
6. Archive a category; confirm it disappears from pickers but remains in reports.
7. Try another user's transaction URL; confirm a 404 page.
8. Navigate everything with keyboard only; test at 360 px width and at 200% zoom; enable reduced motion and confirm no animation.
9. Request a password reset (log mailer) and complete it; delete the account and confirm data is gone.

Responsive, accessibility and security checks

- Lighthouse accessibility 95 or higher on Dashboard, Transactions, Reports
- No console errors; no external network requests apart from the application origin
- Security headers present; cookies are HTTP-only (and secure in production)

- Every state-changing route returns 419 without a CSRF token
- No `{!! !!}` outputs except trusted icons

7. Final Consistency Check (run before finishing)

Audit the finished project against this document set and fix what you find (do not rewrite the documents):

- Every PRD feature F1 to F8 has screens, data and tests (section 5 table)
- Every screen in the App Flow has a route, a view and a test; no extra screens exist
- Every button has a destination and every form has success, error and empty states
- Every table and column in the code matches the Backend Schema names and types
- The same names are used for the same concepts (Naming rules on the instructions page)
- Phases were built in dependency order and nothing outside PRD scope was added
- Permissions: every controller action is covered by a policy or scoped query and a test

Appendix A: Setup, Configuration and CI

Setup commands

```
composer create-project laravel/laravel pocketledger
cd pocketledger
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan laravel/pint
./vendor/bin/pest --init
npm uninstall tailwindcss @tailwindcss/vite axios      # remove whatever the skeleton ships with
npm install chart.js @fontsource-variable/inter @fontsource-variable/fraunces
npm install --save-dev typescript vitest jsdom eslint @eslint/js typescript-eslint
rm -rf resources/js                                    # replaced by resources/ts
cp .env.example .env && php artisan key:generate
```

> The skeleton's `vite.config.ts` or `vite.config.js` must be replaced with the file below and the `@tailwindcss/vite` plugin removed. Delete the default Tailwind imports from `resources/css/app.css`. If a package named above does not exist under that name, use its current equivalent and log it in `docs/DECISIONS.md`.

.env sample (local)

```
APP_NAME=PocketLedger
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost:8000
APP_TIMEZONE=Asia/Kolkata
APP_LOCALE=en
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pocketledger
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=database
SESSION_LIFETIME=120
CACHE_STORE=database
QUEUE_CONNECTION=sync
MAIL_MAILER=log
MAIL_FROM_ADDRESS="noreply@pocketledger.test"
MAIL_FROM_NAME="${APP_NAME}"
VITE_APP_NAME="${APP_NAME}"
```

In `phpunit.xml` set `DB_CONNECTION=mysql`, `DB_DATABASE=pocketledger_test`, `MAIL_MAILER=array`, `SESSION_DRIVER=array`, `CACHE_STORE=array`, `QUEUE_CONNECTION=sync`.

vite.config.ts and tsconfig.json

```
// vite.config.ts
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
  plugins: [
    laravel({
      input: ['resources/css/app.css', 'resources/ts/app.ts'],
      refresh: true,
    }),
  ],
});

// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
```

```
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "isolatedModules": true,
    "skipLibCheck": true,
    "noEmit": true,
    "types": ["vite/client"]
  },
  "include": ["resources/ts/**/*.ts", "vite.config.ts", "vitest.config.ts", "tests/ts/**/*.ts"]
}
```

package.json scripts

```
"scripts": {
  "dev": "vite",
  "build": "tsc --noEmit && vite build",
  "typecheck": "tsc --noEmit",
  "lint": "eslint resources/ts tests/ts",
  "test:ts": "vitest run"
}
```

Blade layout includes assets with `@vite(['resources/css/app.css', 'resources/ts/app.ts'])`. `vitest.config.ts` uses the `jsdom` environment and includes `tests/ts/**/*.test.ts`.

Scheduler registration (routes/console.php)

```
use Illuminate\Support\Facades\Schedule;

Schedule::command('recurring:process')->dailyAt('00:10')->withoutOverlapping();
```

GitHub Actions workflow (.github/workflows/ci.yml)

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: pocketledger_test
        ports: ['3306:3306']
        options: >-
          --health-cmd="mysqladmin ping -h 127.0.0.1 -proot"
          --health-interval=10s --health-timeout=5s --health-retries=10
    env:
      DB_CONNECTION: mysql
      DB_HOST: 127.0.0.1
      DB_PORT: 3306
      DB_DATABASE: pocketledger_test
      DB_USERNAME: root
      DB_PASSWORD: root
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: mbstring, intl, bcmath, pdo_mysql
          coverage: none
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: composer install --no-interaction --prefer-dist
      - run: cp .env.example .env && php artisan key:generate
      - run: npm ci
      - run: npm run typecheck && npm run lint && npm run test:ts
      - run: npm run build
      - run: ./vendor/bin/pint --test
      - run: ./vendor/bin/phpstan analyse --memory-limit=1G
      - run: ./vendor/bin/pest
```

Appendix B: Validation Rules

Implement each as a Form Request. Messages must be plain and specific. Amount strings must match `^\d{1,9}(\.\d{1,2})?$` and be between 0.01 and 999999999.99.

Form	Field rules
Register	• name: required, string, max 100 • email: required, email, max 255, unique:users • password: required, min 8, confirmed • currency_code: required, in INR, USD, EUR, GBP, AED, SGD, AUD, CAD
Login	• email: required, email • password: required • remember: boolean
Forgot and reset	• email: required, email • reset: token required, password min 8 confirmed
Settings profile	• name, email (unique ignoring self), currency_code as in Register
Settings password	• current_password: required, matches the user's password • password: required, min 8, confirmed
Delete account	• password: required, matches the user's password
Category	• name: required, string, max 60, unique per user and type (ignore self) • type: required, in income, expense; cannot change if used • color: required, in the 8 preset hex values
Transaction	• category_id: required, belongs to the user, active (unless it is the transaction's current category) • amount: required, amount-format rule above • transaction_date: required, date, between 2000-01-01 and today • description: nullable, string, max 255
Budgets (bulk)	• month: required, format Y-m, between 2020-01 and current month plus 12 • limits.{category_id}: nullable, amount-format rule; category must be an expense category of the user
Recurring	• category_id: as Transaction • amount: amount-format rule • description: nullable, max 255 • frequency: required, in daily, weekly, monthly, yearly • start_date: required, date, today or later (on create) • end_date: nullable, date, on or after start_date • is_active: boolean
Transaction filters (GET)	• from, to: nullable dates (invalid ignored); type: income or expense; category_id: integer owned by user; q: string max 100; page: integer
Reports month (GET)	• month: format Y-m; invalid falls back to current month
CSV export (GET)	• from: required date; to: required date, on or after from; range at most 5 years

Appendix C: JSON Endpoint Contracts

Both endpoints require authentication, return `application/json` and amounts as decimal strings. Percent values are numbers with one decimal.

GET /reports/data/category-breakdown?month=2026-10

```
{
  "month": "2026-10",
  "currency": "INR",
  "total": "45210.50",
  "items": [
    { "category_id": 3, "name": "Groceries", "color": "#1F5F4D",
      "amount": "12000.00", "percent": 26.5 },
    { "category_id": 7, "name": "Rent", "color": "#B4432F",
      "amount": "10000.00", "percent": 22.1 }
  ]
}
```

Items are sorted by amount descending and include every expense category with spending in the month. The donut chart on the Dashboard groups everything after the sixth item into "Other" on the client.

GET /reports/data/monthly-trend?month=2026-10

```
{
  "currency": "INR",
  "months": [
    { "month": "2026-05", "income": "60000.00", "expense": "41250.00" },
    { "month": "2026-06", "income": "60000.00", "expense": "39800.00" }
  ]
}
```

Always exactly 6 entries ending with the selected month, oldest first. Months without data have `"0.00"`.

Errors

401 when signed out and 404 for unknown routes. Validation of `month` is lenient: invalid values fall back to the current month.

Appendix D: Default Categories and Palette

Type	Categories created at registration (in this order)
Expense	Groceries, Rent, Utilities, Transport, Dining Out, Health, Entertainment, Shopping, Education, Other
Income	Salary, Freelance, Interest, Other Income

Assign colours by cycling the 8 palette colours in the order listed in the UI/UX Brief (pine, brick, slate blue, ochre, plum, teal, umber, olive) across all categories in creation order.

Appendix E: Test Inventory (minimum)

Area	Tests to write
Auth	register seeds defaults and signs in; duplicate email; login ok and fail; throttle; logout; reset flow; guests redirected; 419 redirect
Settings	profile update; wrong current password; delete account removes all owned rows in order
Categories	CRUD; unique per type; archive and restore; delete guard; type lock; foreign 404
Transactions	create, update, delete; validation edges; filters; totals across pages; search escaping; budget message; `return_to` safety; constant query count; foreign 404
Budgets	bulk save; blank removes; copy no-overwrite; status thresholds; month bounds
Dashboard and reports	totals; breakdown; trend zero-fill; top 5; savings rate; JSON shapes; CSV content, BOM and injection prefix; export validation
Recurring	calculator unit tests; catch-up; idempotency; end date; pause and resume; process-due isolation; delete keeps transactions
TypeScript	`formatMoney` (INR grouping, USD); category filter by type; chart data mapping

Appendix F: Glossary

Term	Meaning
Transaction	One recorded income or expense on a date. Direction is determined by its category type.
Category	A user-defined group with a type (income or expense) and a colour.
Budget	A monthly spending limit for one expense category, stored with `month` as the first day of the month.
Recurring transaction	A rule with frequency and start date that generates transactions when due.
Occurrence	One due date of a recurring rule, computed from the start date and the run count.
Budget status	On track (under 80%), Nearing limit (80% to 100%), Over budget (above 100%).
Savings rate	Net divided by income for the month, as a percentage.
Archived category	Hidden from entry pickers but kept in history and reports.

> End of specification. Build phases 0 to 9 in order, verify each phase, then finish with the Final Verification and Final Consistency Check in the Implementation Plan.